

DOCUMENTO ARQUITECTÓNICO MAESTRO

Ecosistema Digital de Gestión y Alquiler de Canchas Deportivas

Versión 2.0 — Edición Profesional Ampliada

Documento de Referencia para Arquitectura, Ingeniería y Desarrollo

Estado: APROBADO PARA IMPLEMENTACIÓN

Audiencia: Arquitectos de Software, Ingenieros, DevOps, QA, Product Managers

Clasificación: Confidencial — Uso Interno

Tabla de Contenido

Tabla de Contenido	2
Control del Documento.....	7
Historial de Revisiones	7
Audiencia y Cómo Leer este Documento.....	7
Glosario de Términos Críticos	7
1. Visión General del Sistema	9
1.1. Visión, Misión y Principios Rectores	9
Visión.....	9
Misión	9
Principios Rectores de Ingeniería.....	9
1.2. Propuesta de Valor por Actor	9
Para el Partner (Dueño del complejo).....	10
Para el Staff (Administrador de turno).....	10
Para el Usuario (Jugador).....	10
Para el Admin de Plataforma	10
1.3. Estrategia de Despliegue Comercial (Go-To-Market)	10
1.4. Restricciones del Proyecto	11
2. Actores y Modelo de Permisos	13
2.1. Definición de Actores.....	13
2.2. Matriz de Permisos (RBAC)	13
3. Arquitectura de Alto Nivel	15
3.1. Vista de Contenedores (estilo C4).....	15
3.2. Patrón Arquitectónico: Hexagonal (Ports & Adapters).....	15
3.2.1. Las tres capas	16
3.2.2. Estructura de carpetas obligatoria.....	16
3.3. Vista de Despliegue Recomendada.....	18
4. Máquina de Estados del Slot.....	19
4.1. Estados Definidos.....	19
4.2. Diagrama de Transiciones	19
4.3. Reglas de Concurrencia.....	20
4.3.1. Doble Bloqueo: Optimista + Distribuido	20
4.3.2. Reversión por TTL.....	21
5. Requerimientos Funcionales.....	22

5.1. Identidad y Autenticación.....	22
RF-01: Autenticación de Usuarios.....	22
RF-02: Recuperación de Contraseña.....	22
5.2. Inventario y Disponibilidad	22
RF-03: Generación de Slots por Ventana Móvil de 30 días.....	23
RF-04: Búsqueda Geoespacial	23
5.3. Reservas y Pagos	23
RF-05: Creación de Booking con Múltiples Slots.....	23
RF-06: Bloqueo Anti-Colisión Distribuido.....	24
RF-07: Confirmación por Webhook (Doble Validación).....	24
RF-08: Reconciliación Activa de Pagos.....	24
5.4. Operación Presencial	25
RF-09: Check-in por QR	25
RF-10: Walk-in (Reserva Presencial)	25
RF-11: Apertura, Cierre y Auditoría de Shift	25
RF-12: Marcado de No-Show	26
5.5. Notificaciones	26
RF-13: Notificaciones Inteligentes Multi-canal	26
6. Requerimientos No Funcionales	27
6.1. Rendimiento.....	27
6.2. Disponibilidad y Confiabilidad	27
6.3. Escalabilidad.....	28
6.4. Seguridad	28
6.5. Mantenibilidad.....	29
6.6. Observabilidad	29
7. Casos de Uso (Flujos Detallados)	30
7.1. UC-01: Onboarding de Partner y Generación de Inventario.....	30
7.2. UC-02: Reserva y Pago (Usuario)	30
7.3. UC-03: Check-in y Cobro de Saldo (Staff).....	31
7.4. UC-04: Reserva Walk-in (Presencial).....	31
7.5. UC-05: Apertura y Cierre de Shift	32
8. Reglas de Negocio Estrictas	33
RN-01: Aislamiento Multi-Cancha.....	33
RN-02: Inmutabilidad del Precio (Snapshot).....	33
RN-03: Independencia Transaccional	33

RN-04: Manejo de Conflictos por Cambio de Horario	33
RN-05: Reembolsos y Costo Hundido	34
RN-06: Política de No-Show	34
RN-07: Cobranza Intransferible.....	34
RN-08: Cero Doble Venta	34
RN-09: Inmutabilidad de Shifts Cerrados.....	34
RN-10: Auditoría Append-Only	34
9. Arquitectura de Datos.....	35
9.1. Convenciones Generales.....	35
9.2. Tablas del Sistema.....	35
9.2.1. users	35
9.2.2. venues	36
9.2.3. venue_staff	36
9.2.4. fields.....	37
9.2.5. operating_hours.....	37
9.2.6. slots	37
9.2.7. bookings	38
9.2.8. booking_slots	39
9.2.9. transactions.....	39
9.2.10. webhook_events.....	40
9.2.11. events (bloqueos no comerciales)	40
9.2.12. shift_logs	41
9.2.13. shift_movements	41
9.2.14. audit_logs (append-only)	41
9.2.15. notifications_log	42
9.3. Diagrama Entidad-Relación (resumen)	42
9.4. Estrategia de Migraciones.....	43
10. Contratos de API	44
10.1. Convenciones Generales.....	44
10.2. Códigos de Estado HTTP.....	44
10.3. Estructura Estándar de Errores	45
10.4. Endpoints Críticos (selección)	45
10.4.1. POST /v1/auth/login	45
10.4.2. GET /v1/venues/search.....	46
10.4.3. POST /v1/bookings (creación).....	46

10.4.4. POST /v1/webhooks/payment.....	47
10.4.5. POST /v1/staff/checkin/{qr_token}/complete.....	47
10.5. Rate Limiting	48
11. Stack Tecnológico.....	49
11.1. Backend.....	49
11.2. Frontend.....	49
11.3. Datos	50
11.4. Servicios Externos	50
11.5. Infraestructura y DevOps.....	51
12. Seguridad	52
12.1. Modelo de Amenazas (Resumen)	52
12.2. Gestión de Secretos	52
12.3. Cumplimiento y Datos Personales	53
13. Resiliencia: Diseñado para el Fallo.....	54
13.1. Servicio Núcleo vs Servicios Auxiliares.....	54
13.2. Patrones de Resiliencia Aplicados.....	54
13.2.1. Timeouts Explícitos	54
13.2.2. Reintentos con Backoff Exponencial.....	55
13.2.3. Circuit Breaker	55
13.2.4. Dead Letter Queue (DLQ).....	55
13.2.5. Idempotencia Universal	55
13.3. Escenarios de Fallo Documentados	56
13.4. Health Checks.....	57
13.5. Plan de Disaster Recovery.....	57
14. Plan de Escalamiento	58
14.1. Hitos de Escalamiento.....	58
14.2. Estrategias por Componente	58
14.2.1. API stateless: escalado horizontal.....	58
14.2.2. Base de datos: lectura/escritura	58
14.2.3. Particionamiento de slots	58
14.2.4. Caché agresivo	59
14.2.5. Workers y colas por prioridad.....	59
15. Observabilidad, Testing y Entrega	60
15.1. Observabilidad: los Tres Pilares	60
15.1.1. Logs estructurados.....	60

15.1.2. Métricas (Prometheus / Datadog)	60
15.1.3. Trazas distribuidas (OpenTelemetry)	60
15.2. Alertas Accionables	60
15.3. Estrategia de Testing	61
15.3.1. Tests Obligatorios	61
15.3.2. Pruebas de Carga	62
15.4. CI/CD	62
15.5. Feature Flags	63
16. Reglas de Oro del Desarrollo	64
Regla 1: El Meridiano Cero	64
Regla 2: Claridad Multi-Cancha	64
Regla 3: UI de Cobranza Inequívoca	64
Regla 4: Cero Cálculos Espaciales en Memoria	64
Regla 5: Doble Validación de Pagos	64
Regla 6: Idempotencia por Defecto	64
Regla 7: Deny by Default	64
Regla 8: Separación Estricta de Capas	64
Regla 9: Construir para el Cambio	65
Regla 10: Lo que no se mide no existe	65
17. Gobernanza del Documento y del Código	66
17.1. Cómo Cambia este Documento	66
17.2. Decisiones Arquitectónicas (ADRs)	66
17.3. Definition of Ready (DoR) para una historia	66
17.4. Definition of Done (DoD) para una historia	66
18. Anexos	67
18.1. Anexo A: Convenciones de Nombres	67
18.2. Anexo B: Lista de Verificación Pre-Producción	67
18.3. Anexo C: Recursos y Lecturas Recomendadas	68
18.4. Anexo D: Cierre	68

Control del Documento

Historial de Revisiones

Toda modificación a este documento requiere actualización de esta tabla. El versionado sigue el esquema MAJOR.MINOR. Un cambio MAJOR implica que las decisiones arquitectónicas previas dejan de ser vinculantes; un cambio MINOR es una aclaración o adición compatible.

Versión	Fecha	Autor	Tipo	Descripción del Cambio
1.0	2025-Q4	Equipo Producto	MAJOR	Versión inicial. Visión, actores, máquina de estados, esquema relacional.
2.0	2026-05	Arquitectura e Ingeniería	MAJOR	Refundación profesional: capas hexagonales, contratos de API, observabilidad, resiliencia, seguridad, DevOps, testing y plan de escalamiento.

Audiencia y Cómo Leer este Documento

Este documento está diseñado para ser leído por distintos perfiles. Cada uno encontrará la información que necesita en las secciones indicadas; sin embargo, las decisiones arquitectónicas y reglas de negocio son de lectura obligatoria para todo el equipo.

Perfil	Secciones de Lectura Obligatoria	Secciones de Lectura Recomendada
Arquitecto / Tech Lead	Todas	—
Backend Engineer	1, 2, 3, 4, 5, 6, 7, 8, 11, 12, 13, 14, 15	9, 10, 16
Frontend Engineer	1, 2, 3, 5, 8, 9, 13, 14, 15	6, 7, 10
DevOps / SRE	1, 11, 12, 13, 14, 15, 16	5, 6, 7
QA Engineer	1, 3, 5, 6, 14, 15	8, 9, 10
Product Manager	1, 2, 5, 6, 16	3, 13

Glosario de Términos Críticos

Este glosario es de cumplimiento obligatorio. Si un término aparece definido aquí, debe usarse exactamente con ese significado en código, comentarios, commits y comunicación.

Término	Definición Operacional
Slot	Bloque atómico de tiempo (típicamente 60 minutos) asociado a una cancha específica. Es la unidad mínima de inventario del sistema.
Booking	Contrato de reserva que agrupa uno o más slots contiguos del mismo usuario. Es el documento de venta.
Walk-in	Reserva creada presencialmente por el Staff, sin uso previo de la app. Pago 100% en caja.
No-Show	Reserva pagada cuyo titular no se presentó dentro de los 15 minutos posteriores al inicio. El anticipo se retiene.
Lock	Bloqueo distribuido en Redis sobre un slot durante el proceso de pago, con TTL de 600 segundos.
Webhook	Notificación HTTP firmada (HMAC-SHA256) emitida por la pasarela hacia el backend, fuente única de verdad de pagos.
Idempotency-Key	Identificador único que permite reintentar una operación sin duplicar efectos secundarios.
Snapshot de precio	Copia inmutable del precio y comisiones vigentes al momento exacto de creación del booking.
Shift	Turno de trabajo de un Staff con apertura, cierre, efectivo esperado y efectivo entregado. Base de la auditoría de caja.
UTC	Tiempo Universal Coordinado. Único uso permitido en backend y base de datos.
DTO	Data Transfer Object. Estructura plana usada exclusivamente entre capas o en respuestas de API.
SLA / SLO	Service Level Agreement / Objective. Compromiso medible de disponibilidad o latencia.

1. Visión General del Sistema

Este sistema es un Ecosistema Digital Intermediario (Marketplace) de tipo dos caras (two-sided marketplace) que conecta en tiempo real la oferta ociosa de complejos deportivos con la demanda inmediata de jugadores. La propuesta diferencial frente a competidores convencionales es la reserva de fricción cero (Real-Time Spontaneity): el usuario puede ver, reservar y pagar una cancha en menos de noventa segundos desde que abre la aplicación.

1.1. Visión, Misión y Principios Rectores

Visión

Convertirnos en la infraestructura digital por defecto del alquiler de espacios deportivos en Latinoamérica, eliminando la fricción operativa para los dueños y la incertidumbre para los jugadores.

Misión

Operar una plataforma transaccional confiable, geoespacialmente consciente, financieramente precisa y operacionalmente resiliente que sostenga simultáneamente miles de complejos deportivos y cientos de miles de reservas mensuales sin degradación de servicio.

Principios Rectores de Ingeniería

Estos principios anteceden a cualquier decisión técnica concreta. Cuando una decisión particular entre en conflicto con uno de estos principios, gana el principio.

- **Correctitud antes que velocidad:** una transacción incorrecta cuesta más que una transacción lenta. La integridad financiera nunca se sacrifica por rendimiento percibido.
- **Estado explícito y único:** todo dato crítico tiene una sola fuente de verdad. No hay datos paralelos en caché que puedan diverger sin un mecanismo de invalidación claro.
- **Fallar de forma predecible:** todo componente externo (pasarela, WhatsApp, FCM, mapas) puede fallar; el sistema debe continuar prestando el servicio núcleo (reserva y check-in).
- **Idempotencia por defecto:** toda operación que mute estado debe poder reintentarse sin producir efectos duplicados.
- **Observabilidad como ciudadana de primera clase:** no hay funcionalidad nueva sin logs estructurados, métricas y trazas. Lo que no se mide, no existe en producción.
- **Construir para el cambio, no para el día uno:** este documento existe precisamente para que los desarrolladores no estén construyendo y destruyendo. Las decisiones aquí descritas son contratos.

1.2. Propuesta de Valor por Actor

La plataforma entrega valor diferenciado a cuatro actores. Cada uno enfrenta un problema distinto y la solución debe verbalizarse en sus términos.

Para el Partner (Dueño del complejo)

- **Digitalización total sin costo inicial.** El Partner no paga setup ni licencias mensuales; la plataforma se monetiza por transacción una vez se demuestre tracción.
- **Eliminación de No-Shows** mediante el cobro obligatorio de un anticipo digital que se retiene en caso de inasistencia.
- **Control de caja unificado:** cada turno de Staff genera un cierre auditable con efectivo esperado vs. entregado, eliminando la sospecha de caja.
- **Analítica histórica:** ocupación por hora, día y cancha; ingresos brutos vs. netos; identificación de horarios muertos para campañas.

Para el Staff (Administrador de turno)

- **Herramienta web rápida (PWA)** que funciona en cualquier teléfono moderno sin descargar app. La fricción operativa al ingresar a un turno debe ser inferior a 10 segundos.
- **Validación de ingresos por QR** que muestra de forma masiva e inequívoca el saldo a cobrar en caja.
- **Registro de Walk-ins** para clientes presenciales que no usaron la app, manteniendo todos los flujos en el mismo sistema.
- **Cuadre de caja aislado por turno:** el Staff sabe exactamente qué dinero le pertenece a su turno y qué debe entregar al cierre.

Para el Usuario (Jugador)

- **Autonomía total:** puede asegurar una cancha en segundos desde el sofá.
- **Disponibilidad real basada en su ubicación,** ordenada por proximidad euclidiana (distancia esférica) y filtrable por deporte y horario.
- **Pago dividido y transparente:** ve exactamente qué es anticipo y qué es saldo a pagar en cancha.
- **Historial completo** de reservas pasadas y futuras, con QR siempre disponible.

Para el Admin de Plataforma

- **Curaduría de Partners:** aprobación de nuevos complejos antes de que sean visibles.
- **Resolución de disputas transaccionales:** herramientas de auditoría para revertir, reasignar o investigar bookings.
- **Configuración centralizada de fees** y reglas globales de la plataforma.

1.3. Estrategia de Despliegue Comercial (Go-To-Market)

La estrategia comercial es deliberadamente secuencial. La arquitectura debe soportar las tres fases sin reescritura, lo cual exige que el motor de comisiones, suscripciones y publicidad esté diseñado desde el día uno aunque su valor se mantenga en cero hasta la fase correspondiente.

Fase	Objetivo	Mecánica de Monetización	Implicación Arquitectónica
1. Penetración	Capturar masa crítica de inventario y usuarios.	Comisión transaccional 0%. Sin suscripciones.	El motor de fees existe y se ejecuta, pero parametrizado a cero. No hay deuda técnica al activarlo.
2. Fidelización	Convertirse en el sistema operativo del Partner.	Sigue 0% en transacción. Cobro opcional por funciones premium (analítica avanzada, WhatsApp marketing).	Aparece el módulo de suscripciones. El RBAC ya soporta features flag por Partner.
3. Monetización	Capturar valor sostenible.	Activación gradual de fees transaccionales (3–7 %). Publicidad geolocalizada. Analítica premium.	Activación de columnas platform_fee y motor de cobro de comisión por evento.

1.4. Restricciones del Proyecto

Toda solución arquitectónica debe respetar estas restricciones. Una propuesta que las viole, por elegante que sea, debe rechazarse.

Tipo	Restricción
Negocio	El Partner no paga setup ni licencias en la fase de penetración.
Negocio	El recaudo en efectivo es responsabilidad del Staff físicamente; la plataforma no maneja efectivo.
Legal	Cumplimiento de protección de datos personales aplicable (Ley 29733 Perú y equivalentes regionales).
Legal	Los datos de tarjetas no transitan ni se almacenan en nuestros servidores. Tokenización vía pasarela certificada PCI-DSS.
Técnica	Toda timestamp en backend y BD debe estar en UTC sin excepciones.
Técnica	Montos financieros se almacenan exclusivamente como DECIMAL(10,2). Nunca FLOAT ni DOUBLE.
Técnica	Cero cálculos espaciales en memoria de aplicación. Toda búsqueda por proximidad usa índices SPATIAL.

Tipo	Restricción
Operativa	La PWA debe funcionar en redes 3G inestables comunes en complejos deportivos del interior.

2. Actores y Modelo de Permisos

2.1. Definición de Actores

El sistema reconoce cuatro actores con responsabilidades estrictamente segregadas. Un mismo usuario humano puede tener varios roles en distintos contextos (un Partner puede operar como Staff en su propio local), pero técnicamente son contextos de autorización independientes y deben evaluarse por separado en cada solicitud.

Decisión arquitectónica: RBAC contextual con scope por venue

El modelo de autorización no es global sino contextualizado. Un usuario tiene un rol base (user, partner, staff, admin) y, adicionalmente, una lista de relaciones contextuales: 'soy Staff del venue X' o 'soy Partner de los venues Y, Z'. Cada operación valida el rol base y, cuando aplica, la pertenencia al venue afectado.

Actor	Acceso	Responsabilidades Clave	Restricciones
Usuario (Cliente)	App Web Pública / iOS / Android	Búsqueda geoespacial, reserva instantánea, pago transaccional, historial, calificación.	No puede ver datos de otros usuarios, ni datos financieros agregados de los Partners.
Partner (Dueño)	Dashboard Web Admin	Configuración de locales y canchas, matrices de horarios y precios, alta de Staff, analítica histórica de sus venues.	Solo puede acceder a venues de los que es propietario. Imposibilidad técnica (no solo lógica) de ver venues ajenos.
Staff (Empleado)	PWA Operativa Móvil	Escaneo de QR (check-in), cobro de saldos en efectivo, creación de Walk-ins, manejo de su turno y cierre de caja.	Solo opera dentro del venue al que está asignado y dentro de un shift abierto. Fuera de turno, sin acceso operativo.
Admin (Plataforma)	Backoffice Interno	Curaduría de Partners, resolución de disputas, configuración de fees, monitoreo de salud del sistema.	Toda acción queda en bitácora inmutable (audit_logs). Nadie, incluido el admin, puede borrar registros financieros.

2.2. Matriz de Permisos (RBAC)

La matriz siguiente es el contrato de autorización. Cualquier endpoint nuevo debe validarse contra esta tabla antes de implementarse. Un permiso ausente es un permiso negado por defecto (deny-by-default).

Operación	User	Partner	Staff	Admin
Buscar canchas (público)	✓	✓	✓	✓
Crear booking propio	✓	—	—	✓
Ver bookings propios	✓	—	—	✓
Crear Walk-in	—	—	✓ (en turno)	✓
Validar QR (check-in)	—	—	✓ (en turno)	✓
Marcar No-Show	—	—	✓ (en turno)	✓
Crear / editar venue	—	✓ (propio)	—	✓
Configurar matriz de horarios	—	✓ (propio)	—	✓
Ver analítica de venue	—	✓ (propio)	Limitada (turno)	✓
Crear / desactivar Staff	—	✓ (propio)	—	✓
Cerrar shift	—	—	✓ (propio)	✓
Aprobar nuevo Partner	—	—	—	✓
Configurar platform_fee	—	—	—	✓
Reembolsar booking	—	✓ (propio, con motivo)	—	✓

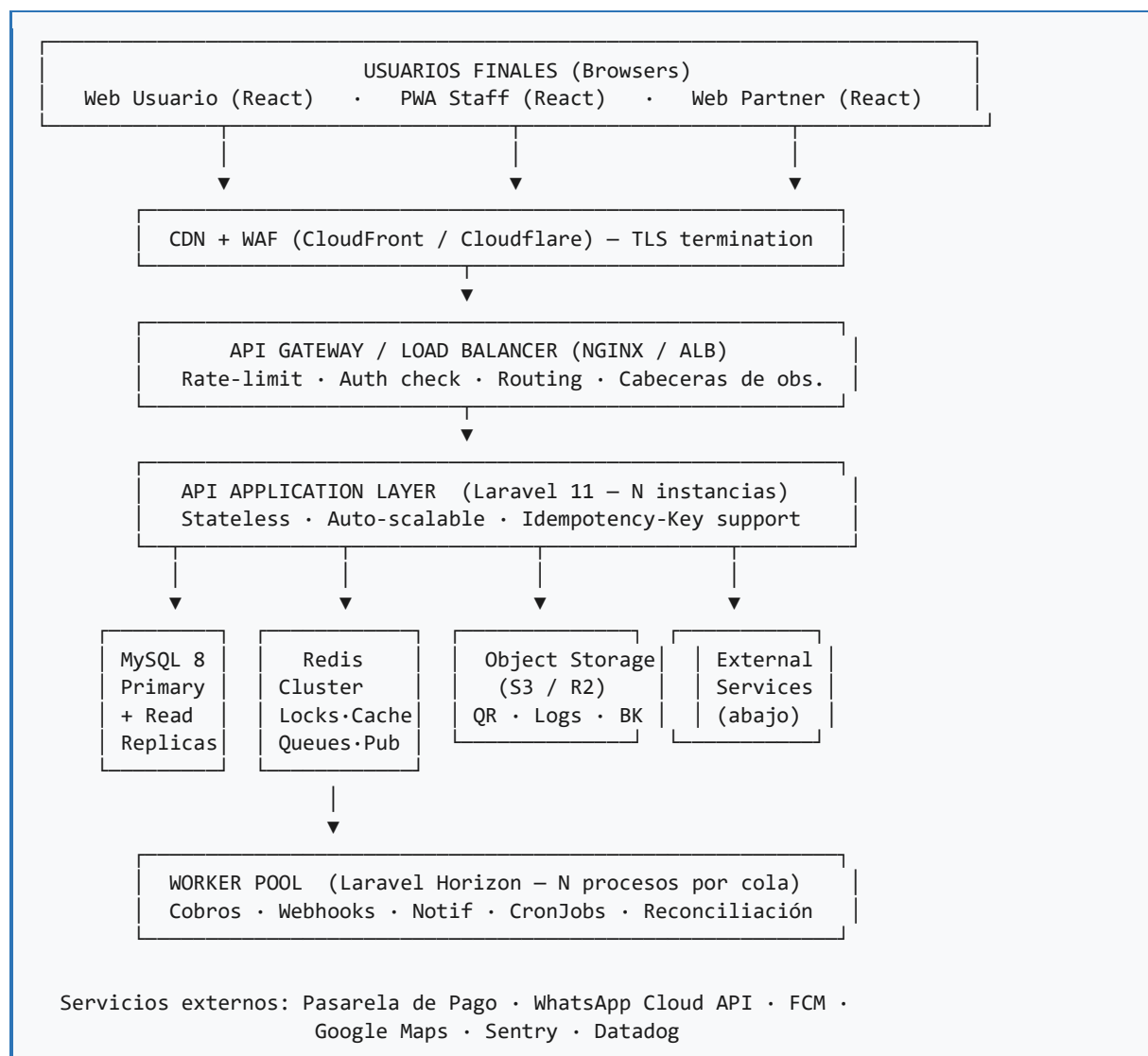
Implementación recomendada

Spatie Laravel Permission para gestión de roles y permisos, combinado con Políticas de Laravel para validación contextual (¿este Staff pertenece a este venue?). Cada Policy debe tener prueba unitaria propia.

3. Arquitectura de Alto Nivel

3.1. Vista de Contenedores (estilo C4)

Esta es la vista de contenedores del sistema. Un contenedor es una unidad desplegable y ejecutable independientemente. La separación que sigue es la mínima necesaria para soportar las cargas previstas y permitir escalamiento horizontal por componente.



3.2. Patrón Arquitectónico: Hexagonal (Ports & Adapters)

El backend se estructura siguiendo Arquitectura Hexagonal con tres capas concéntricas. Esta es la decisión más importante del documento, porque resuelve directamente la queja de fondo: que los desarrolladores

estén constantemente construyendo y destruyendo. La razón habitual de la reescritura es que la lógica de negocio queda atrapada en frameworks o adaptadores; cuando uno cambia, hay que reescribir lo otro.

Por qué hexagonal y no MVC tradicional

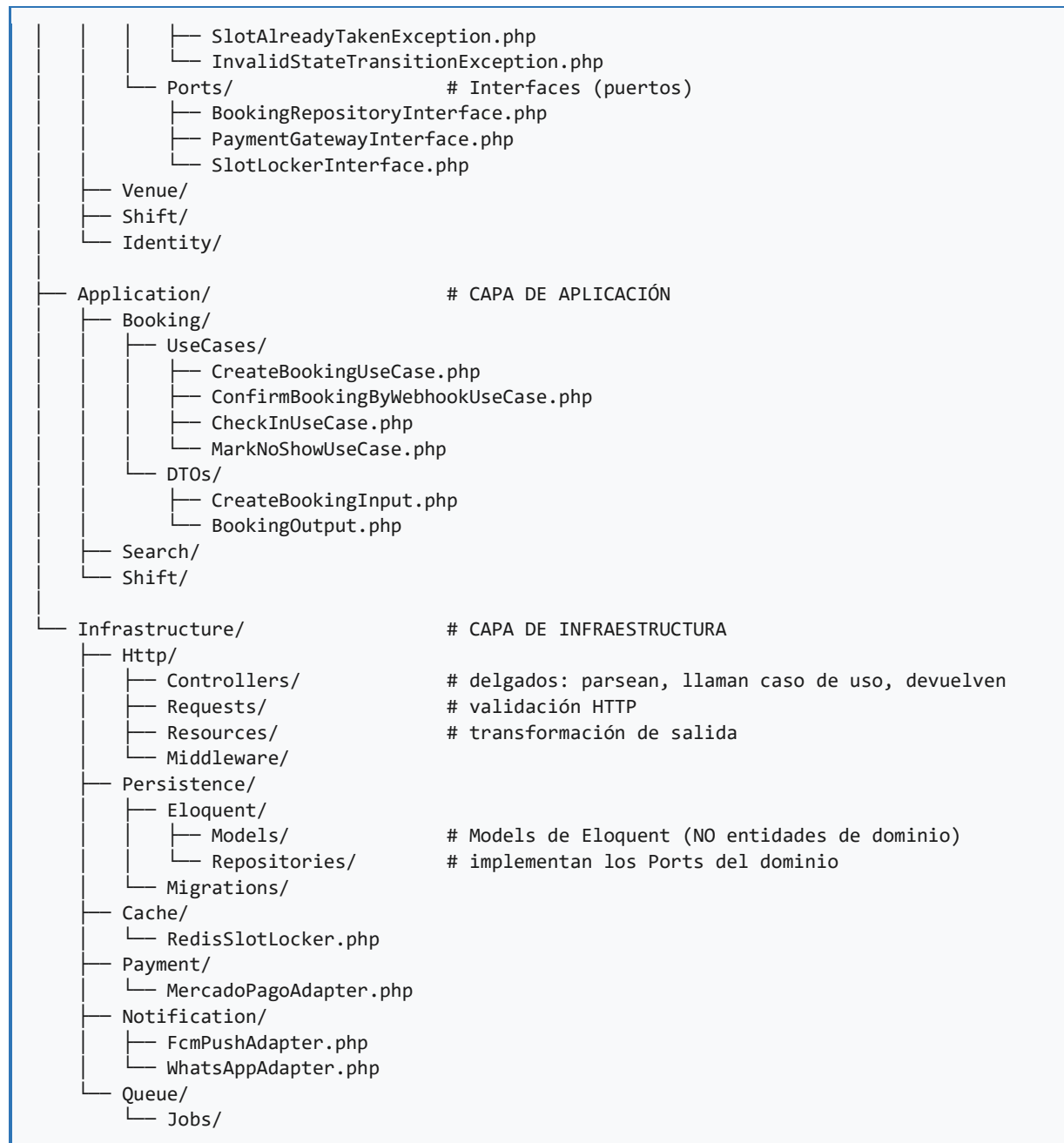
Laravel por defecto invita a poner lógica en Controllers, Models y Services entremezclados. Eso es eficiente al inicio y catastrófico al escalar. La arquitectura hexagonal aísla el dominio (lo que no debe cambiar nunca) de la infraestructura (lo que cambiará: pasarelas, motores de BD, proveedores de notificación). Cuando cambiamos de Mercado Pago a Stripe, no se toca una sola línea de dominio.

3.2.1. Las tres capas

Capa	Responsabilidad	Qué contiene	Qué NO puede hacer
Dominio	Reglas de negocio puras. La verdad inmutable de qué significa una reserva, un slot, un cobro.	Entidades, Value Objects, eventos de dominio, agregados, interfaces (Ports).	No conoce HTTP, Eloquent, Redis, ni framework alguno. Cero importaciones de Laravel.
Aplicación	Orquesta casos de uso. Coordina llamadas al dominio y a los puertos.	Use Cases / Application Services, DTOs de entrada y salida, validaciones de caso de uso.	No accede directamente a base de datos ni a APIs externas. Llama interfaces.
Infraestructura	Implementa los puertos y expone interfaces externas (HTTP, CLI, queue).	Controllers, Repositorios Eloquent, Adaptadores de pasarela, Listeners de webhook, Mailers.	No contiene reglas de negocio. Si tiene un if con lógica de negocio, está en el lugar equivocado.

3.2.2. Estructura de carpetas obligatoria

app/		
├── Domain/		# CAPA DE DOMINIO
│ ├── Booking/		
│ │ ├── Entities/		
│ │ │ ├── Booking.php		# Agregado raíz
│ │ │ └── Slot.php		
│ │ ├── ValueObjects/		
│ │ │ ├── Money.php		# DECIMAL(10,2) tipado
│ │ │ ├── SlotState.php		# enum
│ │ │ └── QRToken.php		
│ │ ├── Events/		
│ │ │ ├── BookingConfirmed.php		
│ │ │ └── BookingNoShowMarked.php		
│ │ └── Exceptions/		



Regla de dependencias (Dependency Rule)

Las flechas de dependencia siempre apuntan hacia adentro: Infraestructura depende de Aplicación, Aplicación depende de Dominio, Dominio no depende de nadie. Si alguna vez un archivo en Domain/ tiene 'use Illuminate\...' o 'use App\Infrastructure\...', el PR se rechaza automáticamente. Esto se valida en CI con un linter de arquitectura (deptrac).

3.3. Vista de Despliegue Recomendada

La topología de despliegue está dimensionada para soportar la fase de penetración (hasta 50.000 usuarios activos mensuales) sin reescritura. El crecimiento posterior se aborda en la sección 13 (Plan de Escalamiento).

Componente	Cantidad	Tamaño Recomendado	Notas
API (Laravel)	2 a 4 instancias	2 vCPU / 4 GB	Stateless. Detrás de balanceador con autoescalado por CPU > 65 %.
Worker (Horizon)	2 instancias	2 vCPU / 4 GB	Procesos separados por prioridad de cola: critical, default, notifications, low.
MySQL primary	1 instancia	4 vCPU / 16 GB / SSD NVMe	Backups continuos (PITR). innodb_buffer_pool_size = 70 % RAM.
MySQL read replica	1 a 2 instancias	4 vCPU / 16 GB	Para búsquedas geoespaciales y reportería. Lag máximo aceptado: 5 segundos.
Redis	1 master + 1 réplica	2 vCPU / 4 GB	AOF habilitado. eviction policy: allkeys-lru para caché, noeviction para queues y locks.
Object Storage	Bucket	Pago por uso	Almacena PDFs de QR, exportes, backups lógicos.
CDN / WAF	Servicio gestionado	—	Cloudflare o CloudFront. Reglas de WAF por OWASP Top 10 activadas.

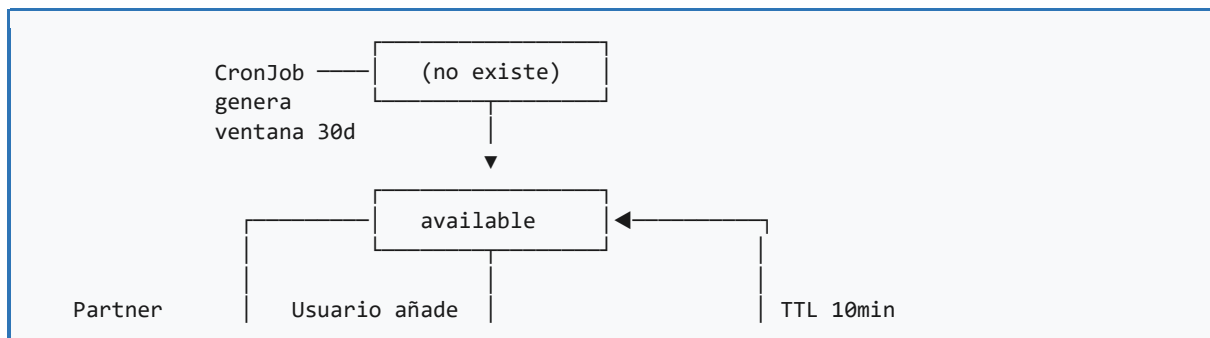
4. Máquina de Estados del Slot

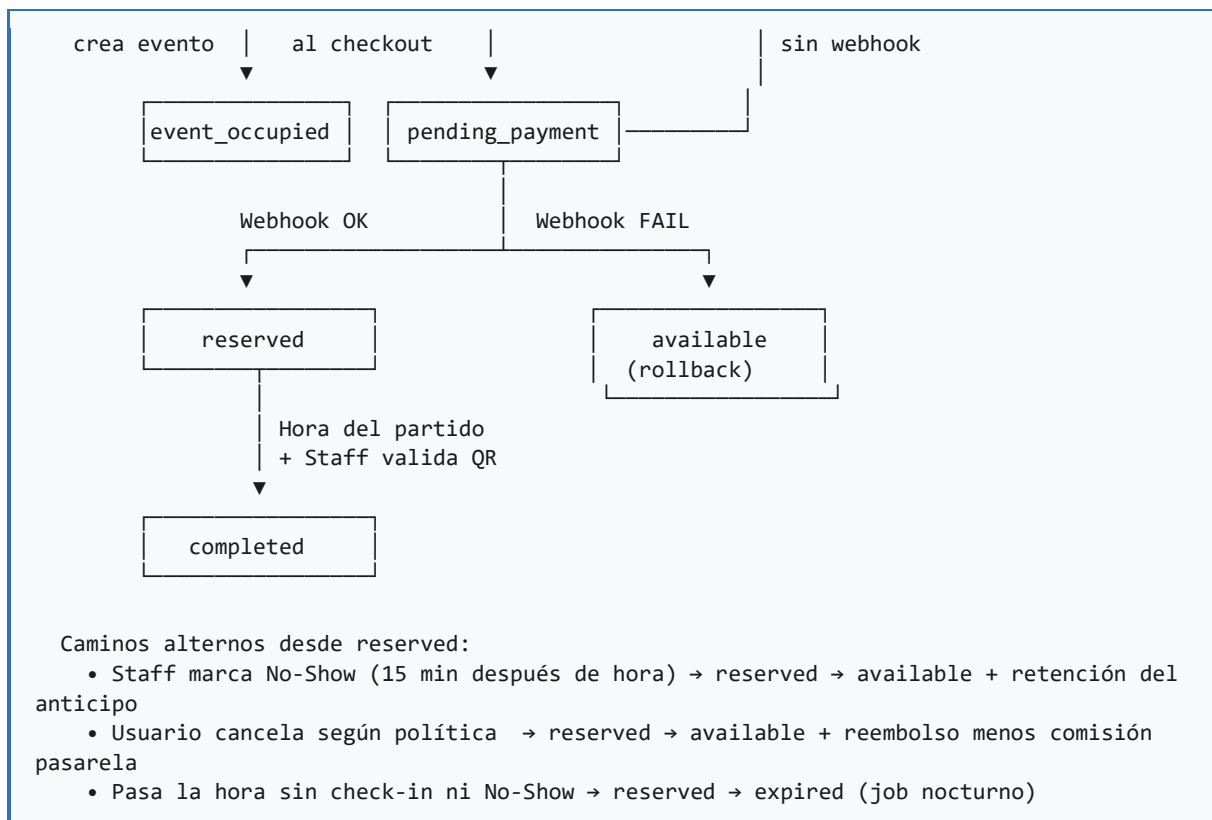
El Slot es la unidad atómica de inventario. Su ciclo de vida está gobernado por una máquina de estados estricta. Cualquier transición no contemplada explícitamente aquí está prohibida y debe lanzar una excepción de tipo `InvalidStateTransitionException`. Esta es la barrera principal contra la inconsistencia de inventario.

4.1. Estados Definidos

Estado	Significado Operacional	Visible al Público	Persistencia
available	Slot libre, puede ser tomado por cualquier usuario.	Sí	MySQL (tabla slots)
pending_payment	Slot bloqueado transitoriamente. Un usuario está pagando. TTL = 600s.	No	MySQL + Lock en Redis con TTL
reserved	Pago confirmado por webhook o reserva manual del Staff (Walk-in). Pertenecer a un Booking.	No (mostrado como ocupado)	MySQL
event_occupied	Bloqueado por el Partner para mantenimiento, torneo, evento privado, etc.	No	MySQL (tabla events) → tapa el slot
completed	El horario del slot ya transcurrió y el booking se cerró correctamente (el partido se jugó).	No	MySQL (estado terminal)
expired	El horario pasó sin que se haya jugado y sin No-Show explícito (caso de borde).	No	MySQL (estado terminal)

4.2. Diagrama de Transiciones





4.3. Reglas de Concurrencia

Es donde más fallan los sistemas de reservas. La doble venta del mismo slot es el escenario catastrófico que debe ser técnicamente imposible, no solo improbable.

4.3.1. Doble Bloqueo: Optimista + Distribuido

Toda transición available → pending_payment se ejecuta dentro de una transacción que combina dos mecanismos:

1. Lock distribuido en Redis con clave 'slot:lock:{slot_id}', TTL de 600 segundos. Implementado con SET NX PX. Si la clave ya existe, otro proceso tiene el slot.
2. Verificación optimista en MySQL: UPDATE slots SET state='pending_payment' WHERE id=? AND state='available'. Si rowsAffected = 0, otro proceso ganó la carrera.
3. Solo si AMBOS mecanismos confirman éxito, la operación procede. De lo contrario, se libera el lock de Redis y se devuelve error 409 Conflict al cliente.

Por qué dos mecanismos en lugar de uno

Redis es rápido pero puede tener fallos transitorios o split-brain en clústeres. MySQL es la fuente de verdad pero su lock pesimista (SELECT FOR UPDATE) tiene costo. Combinar Redis para velocidad y MySQL para durabilidad nos da p99 < 50 ms con garantía de consistencia.

4.3.2. Reversión por TTL

Un job programado (Laravel Scheduler) corre cada 60 segundos buscando slots en pending_payment cuyo lock_expires_at sea menor a la hora actual. Por cada uno, valida activamente con la pasarela (no confía solo en el webhook) si la transacción está pendiente, fallida o exitosa, y aplica la transición correspondiente.

Resiliencia: nunca confiar en una sola fuente

Si el webhook de la pasarela falla o se retrasa, el job de reconciliación consulta la API GET de la pasarela antes de revertir. Esto evita revertir un slot cuyo pago sí ocurrió pero cuyo webhook se perdió.

5. Requerimientos Funcionales

Los requerimientos funcionales describen qué debe hacer el sistema. Cada uno tiene identificador, prioridad (MUST/SHOULD/COULD según MoSCoW), criterios de aceptación verificables y dependencias técnicas. Un RF sin criterios de aceptación es un deseo, no un requisito.

5.1. Identidad y Autenticación

RF-01: Autenticación de Usuarios

Campo	Detalle
Prioridad	MUST
Descripción	Los usuarios pueden registrarse e iniciar sesión por email/password, Google OAuth o Apple Sign-In. Los Partners y Staff usan exclusivamente email/password (sin SSO público).
Mecanismo técnico	Laravel Sanctum (tokens personales para API). Refresh tokens con rotación. Tokens con scope por rol.
Criterios de aceptación	1) Login válido devuelve token y datos de usuario en menos de 400ms. 2) Tres intentos fallidos consecutivos activan rate-limit por 5 minutos. 3) Tokens de Staff expiran al cierre de su shift. 4) Logout invalida el token en backend (no solo en cliente).
Dependencias	—

RF-02: Recuperación de Contraseña

Campo	Detalle
Prioridad	MUST
Descripción	Solicitud de recuperación por email con link de un solo uso, válido por 30 minutos.
Mecanismo técnico	Token aleatorio criptográfico (32 bytes). Hash almacenado en BD (no el token plano).
Criterios de aceptación	1) El link sólo funciona una vez. 2) La respuesta de la API es la misma para emails existentes y no existentes (anti-enumeración). 3) Tras cambiar la contraseña, todas las sesiones existentes se invalidan.

5.2. Inventario y Disponibilidad

RF-03: Generación de Slots por Ventana Móvil de 30 días

Campo	Detalle
Prioridad	MUST
Descripción	Un job diario (03:00 UTC) genera los slots del día 30 a futuro y elimina los slots pasados que nunca fueron reservados (limpieza de inventario muerto).
Mecanismo técnico	Comando Artisan 'slots:roll-window' agendado en Scheduler. Idempotente: ejecutarlo dos veces no duplica slots.
Criterios de aceptación	1) No se calculan disponibilidades al vuelo: la API consulta la tabla 'slots'. 2) Slots reservados no se borran nunca aunque sean pasados (auditoría). 3) Tiempo total del job < 5 minutos para 1.000 venues.
Notas	Si el Partner cambia la matriz horaria, los nuevos horarios solo aplican a slots futuros aún no generados. Slots ya creados respetan la matriz al momento de su creación.

RF-04: Búsqueda Geoespacial

Campo	Detalle
Prioridad	MUST
Descripción	Listado de venues ordenados por proximidad al usuario, con filtros por deporte, fecha y rango horario. Devuelve venues con al menos un slot disponible en la franja solicitada.
Mecanismo técnico	MySQL 8 con columna POINT espacial e índice SPATIAL. Función ST_Distance_Sphere para distancia geodésica.
Criterios de aceptación	1) Respuesta < 300ms (p95) para radios de 25 km. 2) Resultados paginados (20 por página). 3) Caché en Redis del listado por (lat-redondeada, lng-redondeada, deporte) durante 60 segundos.
Restricción crítica	PROHIBIDO calcular distancias en código de aplicación. Toda fórmula geográfica vive en el query SQL.

5.3. Reservas y Pagos**RF-05: Creación de Booking con Múltiples Slots**

Campo	Detalle
Prioridad	MUST

Campo	Detalle
Descripción	Un usuario puede agregar al checkout múltiples slots contiguos del mismo field y pagarlos como un solo booking con un único anticipo.
Restricciones	1) Los slots deben ser del mismo field. 2) Deben ser contiguos sin huecos. 3) Máximo 4 slots (4 horas) por booking.
Criterios de aceptación	1) El precio del booking es la suma de unit_price de cada slot al momento de la reserva (snapshot inmutable). 2) Si uno solo de los slots no puede bloquearse, ningún slot queda bloqueado (transacción atómica). 3) El QR generado es único por booking, no por slot.

RF-06: Bloqueo Anti-Colisión Distribuido

Campo	Detalle
Prioridad	MUST
Descripción	Toda transición available → pending_payment se hace bajo Distributed Lock (Redis) más verificación optimista en MySQL. Ver sección 4.3.
Criterios de aceptación	1) Un test de carga con 200 usuarios concurrentes intentando tomar el mismo slot debe resultar en exactamente 1 reserva exitosa y 199 errores 409. 2) Tiempo medio de la operación de lock < 30ms.

RF-07: Confirmación por Webhook (Doble Validación)

Campo	Detalle
Prioridad	MUST
Descripción	Ningún booking se confirma porque el frontend diga 'pago exitoso'. La fuente de verdad es exclusivamente el webhook firmado HMAC-SHA256 enviado por la pasarela al backend.
Criterios de aceptación	1) Toda solicitud al endpoint de webhook valida la firma; firma inválida → 401 + log de seguridad. 2) El webhook es idempotente: recibir el mismo evento dos veces no produce dos confirmaciones. 3) El webhook responde 200 dentro de 3 segundos; el procesamiento pesado se delega a queue. 4) Cada webhook procesado se almacena en tabla webhook_events para auditoría.

RF-08: Reconciliación Activa de Pagos

Campo	Detalle
Prioridad	MUST
Descripción	Si el webhook se retrasa o pierde, un worker consulta proactivamente a la pasarela el estado de las transacciones en pending_payment cuyo TTL está por expirar.
Criterios de aceptación	1) Job 'payments:reconcile' corre cada 60 segundos. 2) Procesa hasta 100 transacciones por ejecución. 3) Una transacción pagada cuyo webhook se perdió se promueve correctamente a reserved sin intervención humana.

5.4. Operación Presencial

RF-09: Check-in por QR

Campo	Detalle
Prioridad	MUST
Descripción	El Staff escanea el QR del usuario; el sistema muestra Nombre, Cancha, Horario y SALDO PENDIENTE en formato visual masivo. Tras cobrar el efectivo, marca el ingreso como completado.
Criterios de aceptación	1) Tiempo desde escaneo a pantalla de información < 800ms en red 3G. 2) El saldo pendiente está en color rojo y tipografía no menor a 36px. 3) No se permite check-in si el booking no es del venue actual del Staff. 4) No se permite check-in si la hora del slot está más de 30 min en el futuro o más de 60 min en el pasado.

RF-10: Walk-in (Reserva Presencial)

Campo	Detalle
Prioridad	MUST
Descripción	El Staff puede crear una reserva manual para clientes presenciales, marcándola is_walk_in=true. El cobro es 100% en caja, sin anticipo digital.
Criterios de aceptación	1) Solo Staff con shift abierto puede crear walk-ins. 2) El monto se suma directamente al efectivo esperado del shift. 3) No se genera transacción digital ni QR.

RF-11: Apertura, Cierre y Auditoría de Shift

Campo	Detalle
Prioridad	MUST
Descripción	El Staff abre un turno al iniciar su jornada, opera durante el turno y cierra declarando el efectivo entregado. El sistema calcula la diferencia con el efectivo esperado.
Criterios de aceptación	1) Un Staff no puede tener dos shifts abiertos simultáneamente. 2) El cierre requiere ingresar manualmente el efectivo entregado. 3) Si la diferencia es != 0, queda flag de revisión y se notifica al Partner. 4) Una vez cerrado, el shift es inmutable.

RF-12: Marcado de No-Show

Campo	Detalle
Prioridad	MUST
Descripción	Pasados 15 minutos del inicio del slot sin check-in, el Staff puede marcar No-Show. El anticipo se retiene como penalidad y los slots regresan a available para posible Walk-in.
Criterios de aceptación	1) Botón habilitado solo entre min 15 y min 45 desde inicio del slot. 2) Notificación push al usuario informando el cobro de penalidad. 3) Acción reversible solo por Admin con motivo registrado.

5.5. Notificaciones

RF-13: Notificaciones Inteligentes Multi-canal

Campo	Detalle
Prioridad	MUST
Descripción	Envío de notificaciones por Push (FCM) y WhatsApp (Meta API) en momentos clave: confirmación de reserva, recordatorio 1h antes, post-juego (reseña), No-Show.
Filtros antispam	1) Si la reserva es para el mismo día, suprimir la alerta 'mañana tienes partido'. 2) Si el usuario silenció notificaciones para un canal, respetarlo. 3) No más de 3 mensajes por canal por usuario por día.
Criterios de aceptación	1) Las notificaciones se envían vía worker (cola), no en el request HTTP. 2) Si FCM falla, fallback automático a WhatsApp. 3) Toda notificación enviada queda en bitácora notifications_log.

6. Requerimientos No Funcionales

Los requerimientos no funcionales (RNF) describen cómo debe comportarse el sistema. Son los que distinguen un MVP frágil de un producto profesional. Cada uno tiene una métrica medible y un umbral de aceptación. Lo que no se mide, no se cumple.

6.1. Rendimiento

ID	Requerimiento	Métrica	Umbral
RNF-P01	Latencia búsqueda geoespacial	p95 de respuesta API	< 300 ms
RNF-P02	Latencia creación de booking (lock + insert)	p95	< 500 ms
RNF-P03	Latencia check-in QR	p95 desde scan a pantalla	< 800 ms
RNF-P04	Throughput sostenido API	Solicitudes por segundo	≥ 200 rps por instancia
RNF-P05	Tiempo de generación de slots (job nocturno)	Total para 1.000 venues	< 5 minutos
RNF-P06	Time-to-Interactive web Usuario	Lighthouse mobile en 4G	< 3 segundos

6.2. Disponibilidad y Confiabilidad

ID	Requerimiento	Métrica	Umbral
RNF-A01	Disponibilidad mensual API pública	Uptime (excluyendo ventanas de mantenimiento)	≥ 99.9 % (SLA)
RNF-A02	Disponibilidad de check-in (PWA Staff)	Uptime	≥ 99.95 % (operación crítica)
RNF-A03	Recovery Time Objective (RTO)	Tiempo desde caída total a operación restaurada	< 30 minutos
RNF-A04	Recovery Point Objective (RPO)	Pérdida máxima de datos en disaster recovery	< 5 minutos
RNF-A05	Error budget mensual	% de requests que pueden fallar	0.1 % = 43 minutos/mes
RNF-A06	Mean Time To Detection (MTTD)	Desde incidente a alerta	< 2 minutos

6.3. Escalabilidad

ID	Requerimiento	Métrica	Umbral
RNF-E01	Escalado horizontal de API	Sin sesiones en memoria, sin estado local	Cumplido por diseño
RNF-E02	Auto-scaling por carga	Trigger de nuevas instancias	CPU > 65 % por 3 minutos
RNF-E03	Crecimiento de inventario	Slots simultáneos en BD	Hasta 10 millones sin reescritura
RNF-E04	Crecimiento de usuarios	Usuarios concurrentes en pico	5.000 sin degradar p95
RNF-E05	Particionamiento de tablas calientes	slots, bookings, transactions	Particionable por mes (no obligatorio en MVP)

6.4. Seguridad

Los requerimientos de seguridad están alineados con OWASP Top 10 (web) y OWASP API Top 10 (servicios). El detalle operativo está en la sección 12.

ID	Requerimiento
RNF-S01	Tráfico cifrado: solo HTTPS, TLS 1.2+, HSTS habilitado.
RNF-S02	Autenticación con tokens cortos (60 min) y refresh con rotación.
RNF-S03	Tokens revocables del lado del servidor (no JWT puros sin denylist).
RNF-S04	Contraseñas hashadas con bcrypt costo 12 mínimo.
RNF-S05	Webhooks de pasarela validados por firma HMAC. Firma inválida = 401 + alerta.
RNF-S06	Rate-limit por IP y por user_id en endpoints sensibles.
RNF-S07	Validación de input en capa de aplicación (FormRequest) y en dominio (invariantes).
RNF-S08	Logs no contienen PII ni datos financieros completos. Tarjetas: solo last4.
RNF-S09	Auditoría inmutable de operaciones admin (audit_logs append-only).

ID	Requerimiento
RNF-S10	Rotación obligatoria de secretos (DB, pasarela) cada 90 días.

6.5. Mantenibilidad

ID	Requerimiento	Métrica	Umbral
RNF-M01	Cobertura de tests automatizados	Líneas / casos de uso	≥ 70 % línea, 100 % use cases críticos
RNF-M02	Tiempo de build CI completo	Pipeline desde push a verde	< 8 minutos
RNF-M03	Tiempo de despliegue a producción	Desde merge a staging verde	< 15 minutos automatizados
RNF-M04	Documentación de API (OpenAPI)	Endpoints documentados	100 %
RNF-M05	Linter de arquitectura (deptrac)	Violaciones de capas	0 en main

6.6. Observabilidad

ID	Requerimiento
RNF-O01	Logs estructurados en JSON con request_id correlacionable a través de servicios.
RNF-O02	Métricas de negocio expuestas: bookings por hora, walk-ins por shift, % no-show, ingreso/hora.
RNF-O03	Métricas de sistema: latencia por endpoint, RPS, tasa de error 5xx, profundidad de cola.
RNF-O04	Trazas distribuidas (OpenTelemetry) para casos de uso críticos: reserva, check-in, webhook.
RNF-O05	Alertas accionables (no notificaciones), con runbook por cada alerta crítica.

7. Casos de Uso (Flujos Detallados)

Cada caso de uso describe el flujo principal, las precondiciones, postcondiciones, ramas alternas y excepciones. Esta es la fuente única de verdad funcional. Si el código contradice un caso de uso, el código está mal; si el caso de uso es ambiguo, se actualiza este documento antes de codificar.

7.1. UC-01: Onboarding de Partner y Generación de Inventario

Atributo	Detalle
Actor	Partner (asistido por Admin en aprobación)
Precondiciones	El Partner se ha registrado y completado validación KYC básica.
Postcondiciones	Existen registros en venues, fields, operating_hours; se generaron slots para los próximos 30 días.
Flujo principal	1. Partner ingresa datos del local incluyendo lat/lng (capturados con Google Places). 2. Registra cada cancha individual. 3. Define matriz horaria semanal y precio base. 4. Admin aprueba el venue. 5. El sistema dispara job 'venues:generate-slots' que puebla la tabla slots para los próximos 30 días. 6. Notificación push y email al Partner: 'Tu local está activo'.
Excepciones	Lat/lng inválidos → rechazo. Matriz horaria con huecos lógicos → advertencia, no bloqueante.

7.2. UC-02: Reserva y Pago (Usuario)

Atributo	Detalle
Actor	Usuario
Precondiciones	Usuario autenticado, geolocalización autorizada o ubicación manual ingresada.
Postcondiciones (éxito)	Existe un booking en estado reserved con su transaction asociada y QR generado.
Flujo principal	1. Usuario busca canchas (RF-04). 2. Selecciona venue y franja horaria. 3. Marca uno o más slots contiguos. 4. POST /api/bookings con Idempotency-Key. 5. Backend bloquea slots (RF-06). 6. Backend crea booking en pending_payment y transaction en pending. 7. Backend devuelve preference_id de la pasarela. 8. Frontend redirige a pasarela. 9. Usuario paga. 10. Pasarela envía webhook firmado al backend. 11. Backend verifica firma, marca slots reserved, transaction approved, genera QR. 12. Backend dispara notificación push: 'Reserva confirmada'.

Atributo	Detalle
Ramas alternas	RA1: Pasan 10 minutos sin webhook. Job de reconciliación consulta a la pasarela. Si la transacción está aprobada, completa el flujo. Si está fallida o pendiente, libera los slots. RA2: El usuario abandona el checkout. El TTL del lock libera los slots automáticamente. RA3: El usuario cierra y reabre la app durante el pago. La Idempotency-Key evita duplicación.
Excepciones	Pasarela timeout en creación de preferencia → reintento exponencial 3 veces, luego 503 al usuario y rollback. Webhook con firma inválida → 401, no se procesa, alerta a seguridad.

7.3. UC-03: Check-in y Cobro de Saldo (Staff)

Atributo	Detalle
Actor	Staff
Precondiciones	Staff con shift abierto en el venue. Booking en estado reserved del mismo venue.
Postcondiciones	Booking marcado como checked_in. shift_logs.cash_collected actualizado con el saldo cobrado.
Flujo principal	1. Usuario muestra QR. 2. Staff escanea con la PWA. 3. PWA hace GET /api/staff/checkin/{qr_token}. 4. Backend valida que el booking pertenece al venue del shift, que está en ventana válida (-60min a +30min). 5. Pantalla muestra Nombre, Cancha, Horario y SALDO PENDIENTE en grande y rojo. 6. Staff cobra efectivo y presiona 'Completar ingreso'. 7. POST /api/staff/checkin/{qr_token}/complete. 8. Backend transita el booking a checked_in y suma el monto a shift_logs.
Excepciones	QR no pertenece al venue → mensaje 'Reserva de otro local'. Hora fuera de ventana → mensaje claro. Doble escaneo → idempotente, muestra 'ya validado'.

7.4. UC-04: Reserva Walk-in (Presencial)

Atributo	Detalle
Actor	Staff
Precondiciones	Staff con shift abierto. Existe un slot available en el field.
Postcondiciones	Booking creado con is_walk_in=true en estado reserved. Monto sumado al efectivo esperado del shift.

Atributo	Detalle
Flujo principal	1. Staff abre la pestaña 'Walk-in' en la PWA. 2. Selecciona slot libre en una cancha. 3. Ingresar datos opcionales del cliente (nombre, teléfono). 4. POST /api/staff/walk-ins. 5. Backend bloquea el slot, crea booking con is_walk_in=true, monto = unit_price total, paid_advance=0, pending_balance=monto_total. 6. Después del check-in, el cobro completo va al shift como Walk-in cash.
Excepciones	Slot ya tomado durante el flujo → 409, refrescar pantalla.

7.5. UC-05: Apertura y Cierre de Shift

Atributo	Detalle
Actor	Staff
Apertura	1. Staff inicia sesión en PWA. 2. POST /api/staff/shifts/open. 3. Backend valida que no hay otro shift abierto del mismo Staff. 4. Crea registro en shift_logs con started_at=now() y opening_cash (efectivo inicial declarado). 5. Devuelve shift_id que se incluye en todos los headers subsiguientes.
Operación	Durante el shift, todo cobro o walk-in suma al expected_cash.
Cierre	1. Staff presiona 'Cerrar turno'. 2. PWA muestra el efectivo esperado. 3. Staff ingresa el efectivo entregado físicamente. 4. POST /api/staff/shifts/close. 5. Backend calcula difference = declared_cash - expected_cash y guarda. 6. Si difference != 0, marca needs_review y notifica al Partner. 7. El shift queda inmutable.
Postcondiciones	shift_logs cerrado con expected_cash, declared_cash y difference.

8. Reglas de Negocio Estrictas

Las reglas de negocio (RN) son invariantes inmutables del producto. El código es libre de cómo implementarlas, pero no de cumplirlas. Cada RN tiene un test asociado que se ejecuta en CI; si el test falla, el deploy se bloquea.

RN-01: Aislamiento Multi-Cancha

Cada cancha (field) tiene ciclos de vida y disponibilidad totalmente independientes. Si el usuario reserva 'Cancha 1', 'Cancha 2' debe seguir libre. La interfaz, en cualquier punto del flujo (búsqueda, checkout, confirmación, QR, check-in), debe mostrar siempre el nombre explícito de la cancha. Mostrar 'Reserva 19:00' sin especificar cancha es defecto crítico.

Test asociado

tests/Feature/MultiFieldIsolationTest.php verifica que dos bookings simultáneos en canchas distintas del mismo venue no se interfieren.

RN-02: Inmutabilidad del Precio (Snapshot)

Al crear un booking, el unit_price y platform_fee vigentes en ese instante exacto se persisten en columnas dedicadas del booking. Si el Partner sube los precios al día siguiente, las reservas ya hechas no se alteran. La ley general es: 'el precio se cobra al momento de la decisión de compra, no al momento del consumo'.

Aplicación técnica

Las columnas snapshot_unit_price, snapshot_platform_fee y snapshot_total están en la tabla bookings, no se hace JOIN a fields para mostrar el precio histórico.

RN-03: Independencia Transaccional

Un booking puede tener múltiples intentos de pago: el usuario abandona la primera pasarela y vuelve a intentar; la primera transacción rebota y se reintenta. Por tanto, bookings y transactions son tablas en relación 1 a N. Una transacción nunca se borra ni se sobrescribe; cada intento es un registro nuevo.

RN-04: Manejo de Conflictos por Cambio de Horario

Si un Partner edita su matriz horaria (por ejemplo, decide cerrar más temprano) y existen reservas pagadas en el nuevo horario de cierre, el sistema NO ELIMINA las reservas. Dispara una alerta de tipo 'Conflicto operativo' al Partner para que gestione la cancelación o reembolso manualmente. La integridad de la compra del usuario manda sobre la conveniencia operativa del Partner.

Implicación

Tabla 'horario_conflicts' que registra cada caso para auditoría. Notificación push y email al Partner.

RN-05: Reembolsos y Costo Hundido

Las cancelaciones permitidas por el usuario descuentan automáticamente la comisión cobrada por la pasarela de pagos (típicamente 3–5 %). Ni la plataforma ni el Partner asumen pérdidas por arrepentimiento del cliente. El reembolso al usuario es: $\text{monto_pagado} - \text{comisión_pasarela}$.

RN-06: Política de No-Show

Pasados 15 minutos del inicio del slot sin check-in, el Staff puede marcar la reserva como No-Show. Consecuencia: el anticipo se retiene como penalidad (no se reembolsa) y el slot se libera para reventa como Walk-in. Si el Staff no marca No-Show entre los minutos 15 y 45, un job nocturno marca como expired sin penalidad.

RN-07: Cobranza Intransferible

La plataforma calcula deudas con precisión, pero el recaudo físico de efectivo es responsabilidad indelegable del Staff. La plataforma nunca maneja efectivo ni media en disputas presenciales por billetes.

RN-08: Cero Doble Venta

Es técnicamente imposible que dos bookings distintos contengan el mismo slot en estado reserved. Esta invariante está protegida por: (a) constraint UNIQUE en la tabla pivot booking_slots sobre slot_id, (b) lock distribuido durante checkout, (c) verificación optimista en MySQL.

RN-09: Inmutabilidad de Shifts Cerrados

Un shift cerrado no puede modificarse, ni siquiera por Admin. Si hay error, se crea un audit_log con la corrección manual y se anota en notes; los valores originales permanecen como hash de evidencia.

RN-10: Auditoría Append-Only

Toda operación que modifique entidades financieras (bookings, transactions, shifts) genera un registro en audit_logs. Esta tabla solo acepta INSERT; UPDATE y DELETE están prohibidos a nivel de privilegios MySQL.

9. Arquitectura de Datos

Esta sección define el esquema relacional con suficiente precisión para que un DBA o ingeniero de datos pueda generar las migraciones sin tomar decisiones implícitas. Cada tabla declara columnas, tipos exactos, constraints, índices, y propósito arquitectónico.

9.1. Convenciones Generales

- **Motor:** InnoDB en todas las tablas (ACID + foreign keys).
- **Charset:** utf8mb4 con collation utf8mb4_unicode_ci.
- **PK:** BIGINT UNSIGNED AUTO_INCREMENT en todas las tablas, salvo cuando se indique UUID.
- **Timestamps:** created_at y updated_at en todas las tablas (TIMESTAMP en UTC).
- **Soft delete:** deleted_at solo en tablas de configuración (venues, fields, users). Bookings y transactions NUNCA se borran.
- **Money:** DECIMAL(10,2) sin excepciones. Prohibido FLOAT/DOUBLE para dinero.
- **Booleanos:** TINYINT(1) UNSIGNED, default 0.
- **Enums:** se usan ENUM de MySQL para estados con valores cerrados; documentados en este documento.
- **FKs:** ON DELETE RESTRICT por defecto. Solo CASCADE en tablas pivote.

9.2. Tablas del Sistema

9.2.1. users

Identidades de todos los actores. El campo role discrimina el comportamiento.

```
CREATE TABLE users (
  id          BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  uuid        CHAR(36) NOT NULL UNIQUE,          -- expuesto en URLs, no el id
  name        VARCHAR(120) NOT NULL,
  email       VARCHAR(180) NOT NULL UNIQUE,
  email_verified_at TIMESTAMP NULL,
  phone       VARCHAR(20) NULL,
  password    VARCHAR(255) NOT NULL,             -- bcrypt cost 12+
  role        ENUM('user', 'partner', 'staff', 'admin') NOT NULL DEFAULT 'user',
  fcm_token   VARCHAR(255) NULL,
  whatsapp_opt_in TINYINT(1) UNSIGNED NOT NULL DEFAULT 1,
  push_opt_in  TINYINT(1) UNSIGNED NOT NULL DEFAULT 1,
  last_login_at TIMESTAMP NULL,
  created_at   TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at   TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  deleted_at   TIMESTAMP NULL,
  INDEX idx_users_role (role),
```

```
INDEX idx_users_email_verified (email, email_verified_at)
) ENGINE=InnoDB;
```

9.2.2. venues

Complejos deportivos. Contiene la columna espacial location indispensable para búsqueda geográfica.

```
CREATE TABLE venues (
  id          BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  uuid        CHAR(36) NOT NULL UNIQUE,
  partner_id  BIGINT UNSIGNED NOT NULL,          -- FK a users (role=partner)
  name        VARCHAR(180) NOT NULL,
  description  TEXT NULL,
  address     VARCHAR(255) NOT NULL,            -- legible para UI
  city        VARCHAR(120) NOT NULL,
  location    POINT NOT NULL SRID 4326,         -- WGS84 (lat/lng)
  phone       VARCHAR(20) NULL,
  approved_at TIMESTAMP NULL,                  -- NULL hasta aprobación de Admin
  active      TINYINT(1) UNSIGNED NOT NULL DEFAULT 1,
  created_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  deleted_at  TIMESTAMP NULL,
  CONSTRAINT fk_venues_partner FOREIGN KEY (partner_id) REFERENCES users(id),
  SPATIAL INDEX idx_venues_location (location),
  INDEX idx_venues_partner (partner_id),
  INDEX idx_venues_approved_active (approved_at, active)
) ENGINE=InnoDB;
```

Búsqueda por proximidad

Query de referencia: `SELECT id, name, ST_Distance_Sphere(location, POINT(?, ?)) AS distance_m FROM venues WHERE approved_at IS NOT NULL AND active = 1 AND ST_Distance_Sphere(location, POINT(?, ?)) < ? ORDER BY distance_m LIMIT 20.`

9.2.3. venue_staff

Tabla pivote que asigna usuarios con role=staff a venues específicos.

```
CREATE TABLE venue_staff (
  id          BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  venue_id    BIGINT UNSIGNED NOT NULL,
  user_id     BIGINT UNSIGNED NOT NULL,
  active      TINYINT(1) UNSIGNED NOT NULL DEFAULT 1,
  created_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT fk_vs_venue FOREIGN KEY (venue_id) REFERENCES venues(id) ON DELETE CASCADE,
  CONSTRAINT fk_vs_user FOREIGN KEY (user_id) REFERENCES users(id),
  UNIQUE KEY uk_venue_user (venue_id, user_id),
)
```

```
INDEX idx_vs_user (user_id, active)
) ENGINE=InnoDB;
```

9.2.4. fields

Cada cancha física dentro de un venue.

```
CREATE TABLE fields (
  id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  venue_id BIGINT UNSIGNED NOT NULL,
  name VARCHAR(80) NOT NULL,          -- "Cancha 1"
  sport ENUM('futbol5', 'futbol7', 'futbol11', 'padel', 'tenis', 'voley', 'basquet', 'otro') NOT NULL,
  surface ENUM('cesped_natural', 'cesped_sintetico', 'cemento', 'arcilla', 'parquet', 'otro') NOT NULL,
  capacity TINYINT UNSIGNED NOT NULL,
  active TINYINT(1) UNSIGNED NOT NULL DEFAULT 1,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  deleted_at TIMESTAMP NULL,
  CONSTRAINT fk_fields_venue FOREIGN KEY (venue_id) REFERENCES venues(id),
  INDEX idx_fields_venue_active (venue_id, active),
  INDEX idx_fields_sport (sport, active)
) ENGINE=InnoDB;
```

9.2.5. operating_hours

Plantilla semanal de operación. Cada fila representa un rango horario de un día de la semana para una cancha.

```
CREATE TABLE operating_hours (
  id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  field_id BIGINT UNSIGNED NOT NULL,
  weekday TINYINT UNSIGNED NOT NULL,          -- 0=domingo .. 6=sábado
  start_time TIME NOT NULL,                    -- ej. 08:00:00
  end_time TIME NOT NULL,                      -- ej. 23:00:00
  unit_price DECIMAL(10,2) NOT NULL,
  slot_duration_min SMALLINT UNSIGNED NOT NULL DEFAULT 60,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT fk_oh_field FOREIGN KEY (field_id) REFERENCES fields(id) ON DELETE CASCADE,
  INDEX idx_oh_field_weekday (field_id, weekday)
) ENGINE=InnoDB;
```

9.2.6. slots

Tabla CALIENTE del sistema. Es la unidad atómica de inventario. Particionable por mes en el futuro.

```

CREATE TABLE slots (
  id          BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  field_id    BIGINT UNSIGNED NOT NULL,
  starts_at   DATETIME NOT NULL,           -- UTC
  ends_at     DATETIME NOT NULL,           -- UTC
  state       ENUM('available','pending_payment','reserved','event_occupied','completed','expired')
              NOT NULL DEFAULT 'available',
  unit_price   DECIMAL(10,2) NOT NULL,      -- snapshot al generarse
  lock_expires_at DATETIME NULL,           -- usado en pending_payment
  version      INT UNSIGNED NOT NULL DEFAULT 0, -- optimistic locking
  created_at   TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at   TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT fk_slots_field FOREIGN KEY (field_id) REFERENCES fields(id),
  UNIQUE KEY uk_slot_field_start (field_id, starts_at),
  INDEX idx_slots_field_state_starts (field_id, state, starts_at),
  INDEX idx_slots_state_lock (state, lock_expires_at)
) ENGINE=InnoDB;

```

Por qué optimistic locking

La columna 'version' permite UPDATE slots SET state='pending_payment', version=version+1 WHERE id=? AND state='available' AND version=?. Si rowsAffected=0, otro proceso modificó el slot. Es la última línea de defensa contra carreras, junto con el lock distribuido.

9.2.7. bookings

Contrato de reserva. Agrupa uno o más slots y captura el snapshot inmutable del precio.

```

CREATE TABLE bookings (
  id          BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  uuid        CHAR(36) NOT NULL UNIQUE,
  user_id     BIGINT UNSIGNED NULL,         -- NULL para walk-ins anónimos
  venue_id    BIGINT UNSIGNED NOT NULL,
  field_id    BIGINT UNSIGNED NOT NULL,
  state       ENUM('pending_payment','reserved','checked_in','no_show','cancelled','expired','completed')
              NOT NULL DEFAULT 'pending_payment',
  is_walk_in  TINYINT(1) UNSIGNED NOT NULL DEFAULT 0,
  slot_count  TINYINT UNSIGNED NOT NULL,
  starts_at   DATETIME NOT NULL,
  ends_at     DATETIME NOT NULL,
  snapshot_unit_price DECIMAL(10,2) NOT NULL,
  snapshot_total DECIMAL(10,2) NOT NULL,    -- snapshot_unit_price *
  slot_count
  snapshot_platform_fee DECIMAL(10,2) NOT NULL DEFAULT 0.00,
  paid_advance DECIMAL(10,2) NOT NULL DEFAULT 0.00,
  pending_balance DECIMAL(10,2) NOT NULL,   -- snapshot_total - paid_advance
  qr_token     CHAR(64) NULL UNIQUE,        -- HMAC, no autoincremental
  staff_id     BIGINT UNSIGNED NULL,        -- quien validó el QR
  checked_in_at DATETIME NULL,

```

```

no_show_at          DATETIME NULL,
cancelled_at         DATETIME NULL,
cancellation_reason  VARCHAR(255) NULL,
created_at           TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
updated_at           TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
CONSTRAINT fk_bookings_user FOREIGN KEY (user_id) REFERENCES users(id),
CONSTRAINT fk_bookings_venue FOREIGN KEY (venue_id) REFERENCES venues(id),
CONSTRAINT fk_bookings_field FOREIGN KEY (field_id) REFERENCES fields(id),
CONSTRAINT fk_bookings_staff FOREIGN KEY (staff_id) REFERENCES users(id),
INDEX idx_bookings_user (user_id, state),
INDEX idx_bookings_venue_starts (venue_id, starts_at),
INDEX idx_bookings_state (state),
INDEX idx_bookings_qr (qr_token)
) ENGINE=InnoDB;

```

9.2.8. booking_slots

Tabla pivote que enlaza bookings con sus slots. Garantiza que un slot no pueda pertenecer a dos bookings.

```

CREATE TABLE booking_slots (
  id          BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  booking_id  BIGINT UNSIGNED NOT NULL,
  slot_id     BIGINT UNSIGNED NOT NULL,
  CONSTRAINT fk_bs_booking FOREIGN KEY (booking_id) REFERENCES bookings(id) ON DELETE
  CASCADE,
  CONSTRAINT fk_bs_slot FOREIGN KEY (slot_id) REFERENCES slots(id),
  UNIQUE KEY uk_slot_unique (slot_id),    -- INVARIANTE CRÍTICA
  INDEX idx_bs_booking (booking_id)
) ENGINE=InnoDB;

```

Esta es la barrera definitiva contra doble venta

El UNIQUE en slot_id impide a nivel de motor que un mismo slot pertenezca a dos bookings, incluso si la lógica de aplicación tuviese un bug. Es el cinturón y los tirantes simultáneamente.

9.2.9. transactions

Bitácora inmutable de cada interacción con la pasarela de pagos.

```

CREATE TABLE transactions (
  id          BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  uuid        CHAR(36) NOT NULL UNIQUE,
  booking_id  BIGINT UNSIGNED NOT NULL,
  gateway     VARCHAR(40) NOT NULL,           -- ej. 'mercadopago'
  gateway_reference VARCHAR(120) NULL,       -- preference_id
  gateway_transaction_id VARCHAR(120) NULL,  -- payment_id
  amount      DECIMAL(10,2) NOT NULL,

```

```

    currency          CHAR(3) NOT NULL DEFAULT 'PEN',
    status
ENUM('pending','approved','rejected','refunded','partially_refunded','expired','error')
    NOT NULL DEFAULT 'pending',
    status_detail     VARCHAR(255) NULL,
    idempotency_key    VARCHAR(80) NOT NULL,
    paid_at            TIMESTAMP NULL,
    refunded_at        TIMESTAMP NULL,
    raw_response        JSON NULL,                -- guardado completo (sin
PII)
    created_at         TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at         TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    CONSTRAINT fk_tx_booking FOREIGN KEY (booking_id) REFERENCES bookings(id),
    UNIQUE KEY uk_tx_idempotency (idempotency_key, gateway),
    INDEX idx_tx_booking_status (booking_id, status),
    INDEX idx_tx_gateway_ref (gateway_reference),
    INDEX idx_tx_status_created (status, created_at)
) ENGINE=InnoDB;

```

9.2.10. webhook_events

Toda recepción de webhook se persiste antes de procesarse. Soporta idempotencia y debugging.

```

CREATE TABLE webhook_events (
    id                BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    gateway            VARCHAR(40) NOT NULL,
    event_id           VARCHAR(120) NOT NULL,          -- id único de la pasarela
    event_type         VARCHAR(80) NOT NULL,
    signature_valid    TINYINT(1) UNSIGNED NOT NULL,
    payload            JSON NOT NULL,
    processed_at       TIMESTAMP NULL,
    process_error       TEXT NULL,
    attempts           TINYINT UNSIGNED NOT NULL DEFAULT 0,
    created_at         TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UNIQUE KEY uk_gateway_event (gateway, event_id),
    INDEX idx_we_processed (processed_at, attempts)
) ENGINE=InnoDB;

```

9.2.11. events (bloqueos no comerciales)

```

CREATE TABLE events (
    id                BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    field_id          BIGINT UNSIGNED NOT NULL,
    type              ENUM('mantenimiento','torneo','evento_privado','clima','otro') NOT NULL,
    starts_at         DATETIME NOT NULL,
    ends_at           DATETIME NOT NULL,
    notes             VARCHAR(255) NULL,
    created_by        BIGINT UNSIGNED NOT NULL,
    created_at        TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

```



```

CONSTRAINT fk_events_field FOREIGN KEY (field_id) REFERENCES fields(id),
CONSTRAINT fk_events_user FOREIGN KEY (created_by) REFERENCES users(id),
INDEX idx_events_field_range (field_id, starts_at, ends_at)
) ENGINE=InnoDB;

```

9.2.12. shift_logs

Control de caja. Cada turno operativo de un Staff queda auditado aquí.

```

CREATE TABLE shift_logs (
  id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  uuid CHAR(36) NOT NULL UNIQUE,
  user_id BIGINT UNSIGNED NOT NULL,
  venue_id BIGINT UNSIGNED NOT NULL,
  status ENUM('open','closed','flagged') NOT NULL DEFAULT 'open',
  opened_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  closed_at TIMESTAMP NULL,
  opening_cash DECIMAL(10,2) NOT NULL DEFAULT 0.00,
  expected_cash DECIMAL(10,2) NOT NULL DEFAULT 0.00,
  declared_cash DECIMAL(10,2) NULL,
  difference DECIMAL(10,2) NULL,
  notes TEXT NULL,
  CONSTRAINT fk_sh_user FOREIGN KEY (user_id) REFERENCES users(id),
  CONSTRAINT fk_sh_venue FOREIGN KEY (venue_id) REFERENCES venues(id),
  INDEX idx_sh_user_status (user_id, status),
  INDEX idx_sh_venue_opened (venue_id, opened_at)
) ENGINE=InnoDB;

```

9.2.13. shift_movements

Detalle granular de cada movimiento de efectivo dentro de un shift.

```

CREATE TABLE shift_movements (
  id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  shift_id BIGINT UNSIGNED NOT NULL,
  booking_id BIGINT UNSIGNED NULL,
  type ENUM('checkin_balance','walk_in','adjustment') NOT NULL,
  amount DECIMAL(10,2) NOT NULL,
  notes VARCHAR(255) NULL,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_sm_shift FOREIGN KEY (shift_id) REFERENCES shift_logs(id),
  CONSTRAINT fk_sm_booking FOREIGN KEY (booking_id) REFERENCES bookings(id),
  INDEX idx_sm_shift (shift_id, created_at)
) ENGINE=InnoDB;

```

9.2.14. audit_logs (append-only)

```

CREATE TABLE audit_logs (

```

```

id          BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
actor_id    BIGINT UNSIGNED NULL,                -- NULL si es del sistema (job)
actor_role  VARCHAR(20) NOT NULL,
action      VARCHAR(80) NOT NULL,
entity_type VARCHAR(60) NOT NULL,
entity_id   BIGINT UNSIGNED NOT NULL,
before_state JSON NULL,
after_state JSON NULL,
ip          VARCHAR(45) NULL,
user_agent  VARCHAR(255) NULL,
created_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
INDEX idx_al_entity (entity_type, entity_id),
INDEX idx_al_actor (actor_id, created_at)
) ENGINE=InnoDB;
-- Privilegios MySQL: el usuario de la aplicación tiene INSERT pero NO UPDATE ni DELETE.

```

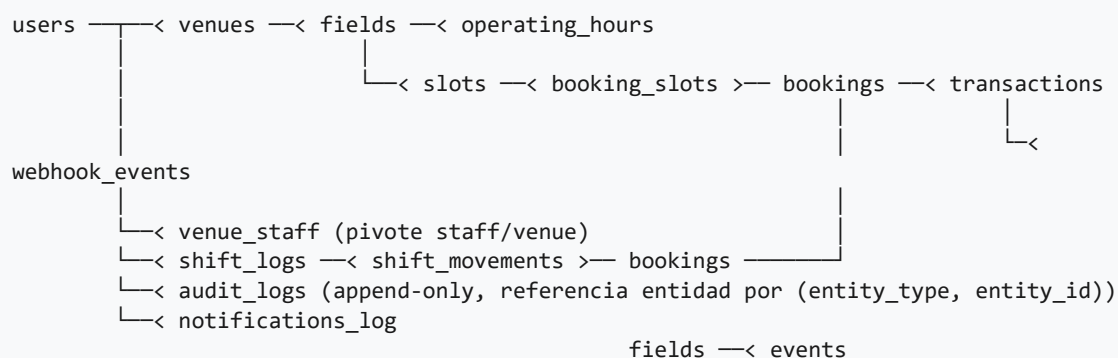
9.2.15. notifications_log

```

CREATE TABLE notifications_log (
  id          BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  user_id     BIGINT UNSIGNED NULL,
  channel     ENUM('push','whatsapp','email','sms') NOT NULL,
  template    VARCHAR(80) NOT NULL,
  payload     JSON NOT NULL,
  status      ENUM('queued','sent','delivered','failed','bounced') NOT NULL,
  provider_msg_id VARCHAR(120) NULL,
  error       VARCHAR(255) NULL,
  sent_at     TIMESTAMP NULL,
  created_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  INDEX idx_nl_user_channel (user_id, channel, created_at),
  INDEX idx_nl_status (status, created_at)
) ENGINE=InnoDB;

```

9.3. Diagrama Entidad-Relación (resumen)



9.4. Estrategia de Migraciones

- **Reversibles:** toda migración debe tener método down. Una migración irreversible requiere aprobación explícita en PR.
- **Sin lock prolongado:** para tablas grandes (slots, bookings), se usa pt-online-schema-change o gh-ost. ALTER TABLE directo está prohibido en producción para tablas con > 1M filas.
- **Datos vs estructura:** las migraciones de estructura y de datos van en archivos separados.
- **Backfill por lotes:** modificar datos masivamente se hace en lotes de 1.000 filas por iteración con SLEEP(0.05) para no saturar la replicación.

10. Contratos de API

Los contratos de API son inmutables una vez liberados. Un cambio breaking requiere versión nueva (v2) y periodo de coexistencia mínimo de 90 días. Esta sección documenta los endpoints más críticos. La especificación OpenAPI completa vive en `/docs/openapi.yaml` del repositorio y es la fuente de verdad.

10.1. Convenciones Generales

- **Base URL:** `https://api.{dominio}/v1`
- **Autenticación:** header `Authorization: Bearer {token}` (Sanctum). Cookies prohibidas en API.
- **Idempotencia:** POST y PUT que muten estado financiero o reservas requieren header `Idempotency-Key` (UUID v4 generado por el cliente).
- **Versionado:** por path (`/v1/`, `/v2/`). Headers de versión solo como complemento.
- **Formato:** JSON con UTF-8. `snake_case` en payloads y query params.
- **Fechas:** siempre ISO 8601 UTC con sufijo 'Z' (2026-05-15T18:00:00Z).
- **Dinero:** string con dos decimales y campo separado de currency. Ejemplo: `{ amount: "45.00", currency: "PEN" }`.
- **Identificadores externos:** siempre uuid en URL pública. El id numérico nunca se expone.

10.2. Códigos de Estado HTTP

Código	Significado	Cuándo se usa
200 OK	Éxito con cuerpo	GET o mutaciones que devuelven el recurso.
201 Created	Recurso creado	POST que creó un recurso. Header <code>Location</code> apunta al recurso.
202 Accepted	Aceptado para proceso asíncrono	POST que delega al worker (notificaciones, reportes).
204 No Content	Éxito sin cuerpo	DELETE u operaciones que no devuelven datos.
400 Bad Request	Payload inválido sintácticamente	JSON malformado, campo de tipo equivocado.
401 Unauthorized	Sin token o token inválido	Token vacío, expirado o revocado.
403 Forbidden	Token válido sin permiso	Permiso denegado por Policy/RBAC.

Código	Significado	Cuándo se usa
404 Not Found	Recurso no existe o no visible para este actor	Incluye casos donde 'no debe saber que existe'.
409 Conflict	Conflicto de estado	Slot tomado por otro, transición inválida, idempotency-key duplicada con payload distinto.
422 Unprocessable Entity	Validación de negocio fallida	Valores correctos en tipo pero inválidos por reglas (ej. slot_count > 4).
429 Too Many Requests	Rate limit	Header Retry-After indica segundos a esperar.
500 Internal Server Error	Error no controlado	Bug. Siempre con error_id correlacionable a logs.
503 Service Unavailable	Dependencia caída	Pasarela inalcanzable, BD en mantenimiento.

10.3. Estructura Estándar de Errores

Toda respuesta de error sigue exactamente este esquema. El frontend depende de él.

```
{
  "error": {
    "code": "SLOT_ALREADY_TAKEN",
    "message": "El horario seleccionado ya no está disponible.",
    "user_message": "Otro jugador tomó este horario hace segundos. Elige otro.",
    "field": "slot_ids",
    "request_id": "req_01J9X7K3F4Z2N5M8P3R6T9V2",
    "documentation_url": "https://docs.api.{dominio}/errors/SLOT_ALREADY_TAKEN"
  }
}
```

Distinción entre message y user_message

El campo 'message' es para desarrolladores y monitoreo. El 'user_message' es lo que la UI muestra al humano. Esto evita que el frontend tenga que mapear códigos a textos amigables y mantiene consistencia entre apps.

10.4. Endpoints Críticos (selección)

10.4.1. POST /v1/auth/login

```
Headers: Content-Type: application/json
Body:    { "email": "user@x.com", "password": "..." }

201 Created:
{
  "token": "1|EXAMPLE...",
  "expires_at": "2026-05-15T19:00:00Z",
  "user": { "uuid": "...", "name": "...", "role": "user" }
}

401 Unauthorized: credenciales inválidas (mensaje genérico anti-enumeración)
429 Too Many Requests: tras 3 intentos fallidos en 5 min
```

10.4.2. GET /v1/venues/search

```
Query params:
lat=number (required)
lng=number (required)
radius_km=number (default 5, max 25)
sport=futbol5|padel|... (optional)
date=2026-05-15 (optional)
start_time=18:00 & end_time=20:00 (optional)
page=1 (default)

200 OK:
{
  "data": [
    {
      "uuid": "...",
      "name": "Complejo Las Palmas",
      "address": "...",
      "distance_m": 1234,
      "fields_count": 3,
      "available_slots_count": 5,
      "thumbnail_url": "..."
    }
  ],
  "meta": { "page": 1, "per_page": 20, "total": 47 }
}

422: lat/lng faltantes o inválidos
503: BD geo en mantenimiento
```

10.4.3. POST /v1/bookings (creación)

```
Headers:
Authorization: Bearer ...
Idempotency-Key: 550e8400-e29b-41d4-a716-446655440000 (REQUIRED)
Content-Type: application/json
```

```

Body:
{
  "slot_uuids": ["uuid-1", "uuid-2"],
  "payment_method": "mercadopago"
}

201 Created (booking creado en pending_payment):
{
  "booking": {
    "uuid": "...",
    "state": "pending_payment",
    "snapshot_total": "90.00",
    "currency": "PEN",
    "expires_at": "2026-05-15T18:10:00Z"
  },
  "payment": {
    "preference_id": "...",
    "init_point": "https://pasarela/checkout/..."
  }
}

```

409 SLOT_ALREADY_TAKEN: alguno de los slots fue tomado entre la búsqueda y este POST
 422 NON_CONTIGUOUS_SLOTS: los slots no son contiguos del mismo field
 422 SLOT_COUNT_EXCEEDED: máximo 4 slots por booking

10.4.4. POST /v1/webhooks/payment

(Endpoint público recibe webhooks firmados de la pasarela)

Headers:
 X-Signature: <HMAC-SHA256(secret, body)>
 X-Event-Id: <id único de la pasarela>

Body: payload de la pasarela

200 OK: webhook aceptado (encolado para procesar)
 401 Unauthorized: firma inválida (alerta a seguridad)
 200 OK con event_already_processed=true si X-Event-Id ya fue visto

Importante: el endpoint NUNCA confía en el payload sin validar firma.
 El procesamiento es asíncrono via cola; el endpoint responde rápido.

10.4.5. POST /v1/staff/checkin/{qr_token}/complete

Headers:
 Authorization: Bearer <token de Staff>
 X-Shift-Id: <uuid del shift abierto> (REQUIRED)
 Idempotency-Key: ... (REQUIRED)

```
200 OK:
{
  "booking": {
    "uuid": "...",
    "state": "checked_in",
    "checked_in_at": "2026-05-15T18:02:11Z",
    "balance_collected": "30.00"
  },
  "shift": {
    "uuid": "...",
    "expected_cash": "180.00"
  }
}
```

403 SHIFT_NOT_OPEN: el staff no tiene shift abierto
403 BOOKING_NOT_IN_VENUE: el booking pertenece a otro venue
409 BOOKING_ALREADY_CHECKED_IN: ya fue validado (idempotente: devuelve 200 si misma key)
409 OUT_OF_TIME_WINDOW: hora fuera de [-60min, +30min]

10.5. Rate Limiting

Implementado en el API Gateway y reforzado en aplicación.

Endpoint	Límite	Ventana	Razón
POST /auth/login	5 / IP	5 minutos	Anti brute-force
POST /auth/forgot-password	3 / email	1 hora	Anti spam
GET /venues/search	60 / user	1 minuto	Carga de mapas
POST /bookings	10 / user	1 minuto	Anti scrape de inventario
POST /webhooks/*	Sin límite por IP de pasarela	—	Necesario para reintentos del proveedor
GET /staff/*	120 / user	1 minuto	Operación intensiva en turno

11. Stack Tecnológico

Cada elección tecnológica está justificada en términos de necesidad concreta. Adoptar tecnología por moda es deuda técnica disfrazada. Los criterios de selección han sido: madurez, ecosistema en español, costo operativo, encaje con el equipo y posibilidad de migración futura sin reescritura.

11.1. Backend

Componente	Elección	Versión Mínima	Justificación
Lenguaje	PHP	8.2	Madurez, tooling de calidad, contratación accesible. Tipos enums y readonly properties.
Framework	Laravel	11.x	Ecosistema completo: Sanctum (auth), Horizon (colas), Telescope (debug), Octane (rendimiento).
Colas	Laravel Horizon sobre Redis	—	Dashboard nativo, prioridades, reintentos, métricas. Suficiente hasta el escalón siguiente.
Cache / Locks / Sesiones (no usadas)	Redis	7.x	TTL nativo, scripts Lua para atomicidad, eviction policies por keypace.
Autenticación	Laravel Sanctum	—	Tokens revocables, simples, scope por habilidad (ability).
Permisos	Spatie Laravel Permission	—	Roles + permisos + Policies; estandarizado.
Tests	Pest + PHPUnit	—	Pest expresivo para tests de feature; PHPUnit para profundidad.
Linter de arquitectura	Deptrac	—	Hace cumplir la regla de dependencias entre Domain/Application/Infrastructure en CI.
Análisis estático	Larastan / PHPStan nivel 8	—	Captura defectos antes de runtime.

11.2. Frontend

Componente	Elección	Versión Mínima	Justificación
Framework	React + Vite	React 18	Ecosistema, contratación, soporte web/móvil con misma base.
Estilos	Tailwind CSS	3.x	Consistencia visual, tamaño de bundle controlado.
Estado	TanStack Query + Zustand	—	TanStack para servidor; Zustand para estado UI local minimalista.
Formularios	React Hook Form + Zod	—	Validación tipada, performance superior a Formik.
PWA	vite-plugin-pwa + Workbox	—	Funcionamiento offline para Staff (check-in básico cacheado).
Mapas	Mapbox GL JS o Google Maps	—	Decisión por costo en producción; ambos contratos abstraídos detrás de un componente.
Tests	Vitest + Testing Library + Playwright	—	Unitarios, integración y E2E.

11.3. Datos

Componente	Elección	Versión Mínima	Justificación
Base de datos primaria	MySQL	8.0	Soporte nativo de SPATIAL, JSON, window functions. Costo operativo conocido.
Cache / Lock / Queue	Redis	7.x	Multifacético; distinguido por DB index dedicado a cada propósito.
Object Storage	S3 (AWS) o R2 (Cloudflare)	—	PDFs de QR, exportes, backups lógicos.
Search avanzado (futuro)	Meilisearch o Elasticsearch	—	Si en fase 2 hay búsqueda por nombre con tolerancia a errores.

11.4. Servicios Externos

Servicio	Proveedor	Uso	Plan B
Pasarela de pagos	Mercado Pago (LATAM)	Cobros con anticipo y reembolso	Stripe configurado como adaptador alternativo; cambio sin reescritura.
WhatsApp	Meta Cloud API	Confirmaciones y recordatorios	Twilio o Wati como respaldo.
Push	Firebase Cloud Messaging	Notificaciones push	OneSignal.
Mapas y Geocoding	Google Maps Platform	Places, Geocoding, Directions	Mapbox + OpenStreetMap (Nominatim) en plan B.
Email transaccional	Postmark o AWS SES	Recuperación de password, comprobantes	Mailgun.
Errores y trazas	Sentry	Captura de excepciones y trazas	—
Métricas y logs	Datadog o Grafana Cloud	Observabilidad operativa	—
Uptime	Better Stack o UptimeRobot	Probes externos	—

11.5. Infraestructura y DevOps

Componente	Elección
Cloud	AWS (ECS Fargate o EC2 con ASG) o equivalente. La arquitectura hexagonal permite portabilidad.
Contenedores	Docker; imágenes multi-stage para minimizar tamaño.
Orquestación	ECS / Fargate en MVP. Migración a EKS si la complejidad lo amerita.
IaC	Terraform en repo separado (infrastructure-as-code/).
CI/CD	GitHub Actions con matrices: lint, tests unit, integration, security scan, build, deploy.
Secretos	AWS Secrets Manager o HashiCorp Vault. NUNCA en .env de Git.
DNS	Route 53 o Cloudflare DNS.
CDN / WAF	CloudFront + AWS WAF, o Cloudflare con managed rules OWASP.

12. Seguridad

La seguridad no es una sección al final del proyecto sino una cualidad transversal. Esta sección consolida las defensas del sistema en capas (defense-in-depth).

12.1. Modelo de Amenazas (Resumen)

Amenaza	Vector	Mitigación principal
Doble venta de slot	Race condition en checkout	Lock distribuido + UNIQUE en booking_slots + optimistic locking
Confirmación falsa de pago	Cliente envía 'pagué'	Webhook firmado HMAC como única fuente de verdad
Webhook falso	Atacante envía POST al endpoint público	Validación HMAC; firma inválida = 401 + alerta
Brute-force credenciales	Diccionario contra /auth/login	Rate-limit + logout + bcrypt costo 12
Enumeración de usuarios	Distinguir email registrado	Mensajes idénticos en login y forgot-password
IDOR (Insecure Direct Object Reference)	Cambiar uuid en URL para ver datos ajenos	Políticas que validan ownership en cada acceso
SQLi	Input no parametrizado	Eloquent + query bindings + análisis estático
XSS	Input renderizado en UI	React escapa por defecto; CSP estricta
CSRF	—	API stateless con tokens Bearer; cookies prohibidas
Robo de QR (replay)	Capturar y reutilizar QR ajeno	QR contiene HMAC; check-in valida venue del staff y ventana temporal
Filtrado de PII en logs	—	Linter custom que prohíbe loguear campos blacklisted (email, phone, address)
Privilegios excesivos del Admin	—	Toda acción Admin va a audit_logs append-only

12.2. Gestión de Secretos

- **Cero secretos en código fuente.** Pre-commit hook con git-secrets/gitleaks bloquea claves.

- **Inyección por entorno:** los secretos se inyectan como variables de entorno desde Secrets Manager al arrancar el contenedor.
- **Rotación automatizada:** secretos de pasarela y BD rotan cada 90 días sin downtime gracias a soporte multi-secreto durante la transición.
- **Acceso de personas:** ningún humano tiene acceso a secretos de producción excepto Tech Lead y Security Lead. Rotación post-acceso humano.

12.3. Cumplimiento y Datos Personales

La aplicación procesa datos personales (nombre, email, teléfono, ubicación). Cumplimiento mínimo:

- **Consentimiento explícito** para términos y privacidad al registrarse.
- **Derecho al olvido:** endpoint de eliminación de cuenta que anonimiza el user_id en bookings históricos (no los borra: integridad financiera).
- **Portabilidad:** endpoint que entrega exporte JSON de los datos del usuario.
- **Minimización:** no se piden datos no requeridos (ej. fecha de nacimiento no se usa, no se pide).
- **PCI:** ningún dato de tarjeta toca nuestros sistemas. Solo recibimos token y last4 desde la pasarela.

13. Resiliencia: Diseñado para el Fallo

Lo que distingue un sistema profesional es lo que ocurre cuando algo falla. Esta sección documenta cómo cada dependencia externa puede caer y qué hace el sistema en ese caso. La premisa: ninguna dependencia externa puede tirar abajo el servicio núcleo.

13.1. Servicio Núcleo vs Servicios Auxiliares

Definimos tres niveles. Un nivel cae sin tirar el siguiente.

Nivel	Funcionalidad	Si falla
Crítico (P0)	Login, búsqueda, check-in QR, cierre de shift	Incidente mayor. Todos despiertan.
Importante (P1)	Crear booking nuevo (nuevas reservas)	Mostrar mensaje degradado: 'Estamos teniendo problemas con pagos. Intenta en minutos.' Bookings existentes no se afectan.
Auxiliar (P2)	Notificaciones push/WhatsApp, analítica, mapa thumbnail	Funcionalidad oculta o degradada. Sin alertas críticas.

13.2. Patrones de Resiliencia Aplicados

13.2.1. Timeouts Explícitos

Toda llamada a servicio externo tiene timeout configurado. Sin excepciones. Por defecto:

Servicio	Connect timeout	Read timeout
Pasarela de pagos (POST preferencias)	3s	10s
Pasarela (GET reconciliación)	3s	8s
WhatsApp Cloud API	2s	5s
FCM	2s	5s
Google Maps Geocoding	2s	4s
MySQL (read)	1s	3s
MySQL (write)	2s	8s
Redis	200ms	500ms

13.2.2. Reintentos con Backoff Exponencial

Estrategia estándar:

- Intento 1: inmediato
- Intento 2: $t = 1s + \text{jitter}(0..200ms)$
- Intento 3: $t = 4s + \text{jitter}(0..500ms)$
- Intento 4: $t = 16s + \text{jitter}(0..1s)$
- Más allá: a Dead Letter Queue (DLQ) para revisión manual

Aplicable a: notificaciones, webhooks salientes, reintento de GET activos contra pasarela.

NO aplicable a: operaciones del usuario en su request HTTP. Para el usuario, una operación falla rápido y el reintento es suyo.

13.2.3. Circuit Breaker

Para evitar bombardear servicios caídos, los adaptadores de pasarela y WhatsApp implementan un circuit breaker: tras N fallos consecutivos en una ventana de tiempo, el circuito se abre y las llamadas siguientes fallan rápido sin red, durante un cooldown. Después intenta una petición de prueba (half-open) y si funciona, vuelve a cerrarse.

Configuración por adaptador:

Pasarela: threshold = 5 fallos / 60s, cooldown = 30s
 WhatsApp: threshold = 10 fallos / 120s, cooldown = 60s
 FCM: threshold = 10 fallos / 120s, cooldown = 60s
 Maps: threshold = 8 fallos / 60s, cooldown = 30s

Estado expuesto en /health/dependencies para dashboards.

13.2.4. Dead Letter Queue (DLQ)

Toda operación encolada que falla más de 4 veces va a una cola DLQ específica del dominio. Las DLQ tienen alertas de profundidad: cualquier elemento en una DLQ dispara notificación en horas hábiles, y revisión manual.

13.2.5. Idempotencia Universal

Toda operación de mutación financiera o de inventario es idempotente. Si el cliente reintenta el mismo POST con el mismo Idempotency-Key:

4. Si la primera operación tuvo éxito, devuelve el mismo resultado sin volver a ejecutar.
5. Si la primera está aún en curso (retry agresivo del cliente), espera con un lock corto (max 2s) y devuelve el resultado o 409 con cuerpo idempotency_in_progress.
6. Si la primera falló, ejecuta normalmente.

7. Si llega un POST con la misma Idempotency-Key pero distinto payload, devuelve 409 IDEMPOTENCY_KEY_MISMATCH.

13.3. Escenarios de Fallo Documentados

Escenario	Detección	Comportamiento del Sistema
Pasarela responde lento (> 10s)	Timeout	Para creación de booking: 503 al usuario con mensaje 'estamos teniendo problemas con pagos, intenta en un par de minutos'. Slot bloqueado se libera automáticamente. Para reconciliación: el job pausa y reintenta.
Pasarela inalcanzable (DNS/red)	Connect timeout / DNS failure	Circuit breaker abre. Frontend muestra banner: 'Pagos temporalmente no disponibles. Reservas existentes no afectadas.' Check-in y otros flujos siguen normales.
Webhook se pierde (3 retries de la pasarela fallaron)	Job reconcile-payments detecta tx pending por > 11 min	Worker consulta GET pasarela/payments/{id}. Si approved, completa el flujo. Si rejected, libera slot y notifica al usuario.
Webhook duplicado por pasarela	X-Event-Id ya existe en webhook_events	Devuelve 200 con event_already_processed=true. No re-procesa.
Redis caído (master)	Health check falla	Failover a réplica. Si toda la instancia muere: locks no funcionan; el sistema rechaza nuevas reservas con 503 (no compromete consistencia). Check-in y consultas siguen porque MySQL sigue de pie.
MySQL primary caído	Connection error en escritura	Frontend recibe 503 en mutaciones. Lecturas pueden seguir desde réplica. Failover a réplica nueva primary en < 60s. Datos no se pierden gracias a binlog.
Worker pool sin capacidad	Profundidad de cola > 1000	Alerta P1. Auto-scaling de workers. Notificaciones se atrasan; bookings funcionan.
Pico de tráfico (10x normal)	RPS > 200/instancia	Auto-scaling crea instancias adicionales en 2-3 min. Mientras tanto, rate-limit por IP protege la integridad.

Escenario	Detección	Comportamiento del Sistema
Bug en una versión nueva	Aumento de errores 500 post-deploy	Sistema de despliegue blue-green permite rollback en < 2 min. Feature flags permiten apagar funcionalidades específicas sin deploy.

13.4. Health Checks

```
GET /health/live → 200 OK si el proceso está corriendo (Kubernetes liveness)
GET /health/ready → 200 OK si la aplicación puede recibir tráfico
                    - DB primary alcanzable
                    - Redis alcanzable
                    - Migraciones al día
                    503 si alguna falla

GET /health/dependencies (admin only)
{
  "database": "ok", "redis": "ok",
  "payment_gateway": "circuit_open",
  "whatsapp": "ok", "fcm": "degraded"
}
```

13.5. Plan de Disaster Recovery

Concepto	Valor
RPO (Recovery Point Objective)	5 minutos máximo de pérdida
RTO (Recovery Time Objective)	30 minutos para servicio núcleo restaurado
Backups automáticos MySQL	Snapshots cada 6h + binlog continuo (PITR)
Backup retention	Diarios 30 días, semanales 90 días, mensuales 1 año
Restore drill	Trimestral. Se documenta tiempo real de restauración.
Replicación cross-region (futuro)	Fase 3 cuando ingresos lo justifiquen

14. Plan de Escalamiento

La arquitectura está dimensionada para soportar la fase MVP sin reescritura. Esta sección documenta cuándo y cómo escalar cada componente. El criterio: escalar antes del dolor, pero no por anticipación supersticiosa.

14.1. Hitos de Escalamiento

Hito	Volumen	Acción
MVP (presente)	≤ 10K usuarios activos / 50K bookings/mes	Topología base de la sección 3.3
H1 (3-6 meses)	10K – 50K usuarios / 250K bookings/mes	Auto-scaling activo, segunda read replica MySQL, Redis cluster con réplica.
H2 (6-12 meses)	50K – 200K usuarios / 1M bookings/mes	Particionar tabla slots por mes. CDN para assets dinámicos. Elasticsearch para búsqueda.
H3 (12+ meses)	200K+ usuarios / 1M+ bookings/mes	Microservicios selectivos: extraer Notifications y Analytics. Multi-región active-passive.

14.2. Estrategias por Componente

14.2.1. API stateless: escalado horizontal

Las instancias API son stateless por diseño: ninguna información de sesión vive en memoria local. Cualquier request puede ir a cualquier instancia. Escalado horizontal por CPU > 65 % durante 3 minutos. Despliegue con conexión drenada (graceful shutdown).

14.2.2. Base de datos: lectura/escritura

MySQL primary recibe escrituras; las read replicas reciben lecturas pesadas (búsqueda geo, reportería).

Estrategia:

- Búsqueda geoespacial → read replica
- Listados de venues, fields, slots → read replica
- Reportes y analítica → read replica dedicada o data warehouse
- Escrituras de booking, transaction, shift → primary
- Lectura tras escritura del mismo usuario (read-after-write) → primary

Implementación: dos conexiones distintas en Laravel (mysql/primary y mysql/replica). El repositorio decide cuál usar. Operaciones financieras siempre primary.

14.2.3. Particionamiento de slots

Cuando la tabla slots supere 5 millones de filas activas, se particiona por mes:

```
ALTER TABLE slots
  PARTITION BY RANGE COLUMNS(starts_at) (
    PARTITION p202605 VALUES LESS THAN ('2026-06-01'),
    PARTITION p202606 VALUES LESS THAN ('2026-07-01'),
    ...
  );
```

Beneficios:

- DROP PARTITION para purgar slots históricos sin LOCK prolongado
- Queries de búsqueda con filtro de fecha tocan menos páginas
- Backups por partición (más rápidos)

14.2.4. Caché agresivo

Datos cacheables en Redis con TTLs deliberados:

Dato	Clave	TTL
Búsqueda de venues por ubicación	search:lat:lng:sport	60s
Detalle de venue público	venue:{uuid}	300s
Disponibilidad de slots por venue/día	slots:venue:{uuid}:date:{date}	30s
Configuración global (fees, etc.)	config:global	300s + invalidación al editar
Tokens de Sanctum (revocación)	token:{hash}	TTL del token

14.2.5. Workers y colas por prioridad

Cola "critical": webhooks de pasarela, confirmaciones (4 workers dedicados)
 Cola "default": jobs de aplicación general (4 workers)
 Cola "notifications": push, WhatsApp, email (8 workers, alta concurrencia)
 Cola "low": reportes, exports (1-2 workers, no urgente)
 Cola "scheduled": cron jobs disparados por scheduler (1-2 workers)

Cada cola escala independientemente. Una avalancha de notificaciones NO retrasa la confirmación de un pago.

15. Observabilidad, Testing y Entrega

15.1. Observabilidad: los Tres Pilares

Sin observabilidad, un sistema de producción es una caja negra. Tres pilares se implementan desde el día uno: logs estructurados, métricas e inspecciones (trazas).

15.1.1. Logs estructurados

Formato JSON obligatorio. Campos mínimos:

```
{
  "timestamp": "2026-05-15T18:00:00.123Z",
  "level": "info|warn|error",
  "request_id": "req_01J9X7K3...",
  "user_id": 1234,           // si aplica
  "actor_role": "user|partner|staff|admin|system",
  "context": "booking.create",
  "message": "human readable",
  "data": { /* sin PII */ }
}
```

PROHIBIDO loggear: passwords, full email (use hash), full phone, tarjetas (full_pan ni siquiera tokenizada), tokens de auth.

Linter custom rechaza commits que loguean campos de la denylist.

15.1.2. Métricas (Prometheus / Datadog)

Tres familias de métricas:

- **Sistema (RED):** Rate (RPS), Errors (% 5xx), Duration (p50/p95/p99) por endpoint.
- **Recursos (USE):** Utilization, Saturation, Errors de CPU, RAM, IO, conexiones DB, profundidad de cola.
- **Negocio:** Bookings creados/hora, % no-show, ingreso bruto por hora, walk-ins por shift, tiempo medio de check-in.

15.1.3. Trazas distribuidas (OpenTelemetry)

Casos de uso críticos están instrumentados con trazas: una sola petición se sigue desde Frontend → API → DB/Redis/Pasarela. Cada span tiene atributos (booking_id, slot_id, etc.) anonimizados.

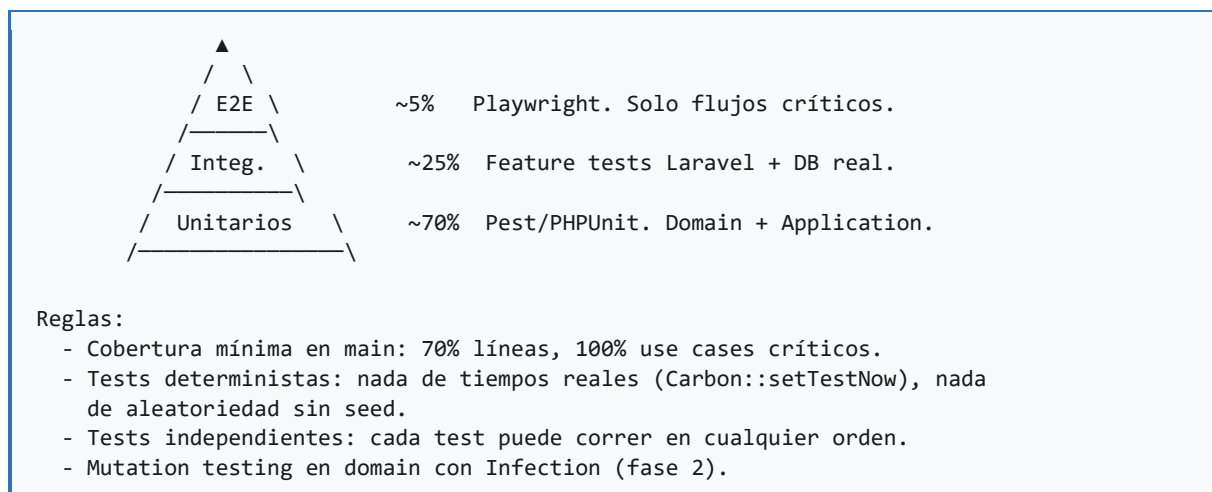
15.2. Alertas Accionables

Una alerta sin acción definida es ruido. Toda alerta tiene un runbook asociado en /docs/runbooks/.

Alerta	Severidad	Trigger	Runbook
API 5xx > 1 % por 5 min	P1	Datadog	runbooks/api-5xx.md
Latencia p95 búsqueda > 500ms por 10 min	P2	Datadog	runbooks/search-latency.md
DLQ con elementos	P2	Horizon	runbooks/dlq-review.md
Circuit breaker pasarela abierto	P1	Custom	runbooks/payment-gateway-down.md
MySQL replication lag > 10s	P1	RDS / Datadog	runbooks/replica-lag.md
Disk usage > 80 %	P2	Cloud monitor	runbooks/disk-pressure.md
Webhook firma inválida > 5 en 1 min	P1 seguridad	Custom	runbooks/webhook-attack.md
Shift cerrado con difference != 0	Operativa	Job	runbooks/cash-discrepancy.md

15.3. Estrategia de Testing

La pirámide de tests se respeta: muchos unitarios rápidos, suficientes de integración, pocos E2E lentos.



15.3.1. Tests Obligatorios

- **Domain** (unitario): cada entidad y value object 100 % cubierto.
- **Application** (unitario con mocks de puertos): cada use case con flujo principal y todas sus excepciones.

- **Infrastructure** (integración con BD real): repositorios verifican que los queries hacen lo que dicen.
- **Concurrencia** (integración pesada): el test de '200 usuarios al mismo slot, exactamente 1 gana' debe existir.
- **Idempotencia** (integración): doble POST con misma key produce un solo efecto.
- **Webhook** (integración): firma válida procesa, firma inválida 401, evento duplicado idempotente.
- **E2E** (Playwright): registro → buscar → reservar → pagar (sandbox) → check-in (Staff).

15.3.2. Pruebas de Carga

Antes de cada release mayor, se corre k6 con escenarios:

- Búsqueda geoespacial: 100 RPS sostenido durante 10 min.
- Checkout concurrente: 200 RPS al mismo venue durante 2 min.
- Webhook flood: 500 webhooks en 30 segundos.
- Pico realista: rampa de 0 a 1000 RPS en 5 min y sostenido 10 min.

15.4. CI/CD

El pipeline es la primera línea de defensa de la calidad. Si rompe, el deploy se bloquea. Sin excepciones.

Pipeline (GitHub Actions):

```
on: pull_request, push:main

job: lint
  - php-cs-fixer (formato)
  - phpstan / larastan nivel 8
  - deprac (arquitectura: dominio no depende de framework)

job: tests-unit (paralelo en matriz)
  - Pest unitarios + cobertura

job: tests-integration
  - levanta MySQL, Redis en services
  - corre tests/Feature
  - corre tests/Concurrency

job: security
  - composer audit
  - npm audit
  - gitleaks (secretos)
  - trivy (imagen Docker)
```

```
job: build
- Docker multi-stage
- sube imagen con tag = git sha

job: deploy-staging (auto en main)
- aplica migraciones (con dry-run primero)
- blue-green: levanta versión nueva, smoke tests, switch
- smoke E2E en staging

job: deploy-prod (manual con approval)
- mismo flujo blue-green
- feature flags por defecto en off para nuevas features
- rollback automático si error rate > 1% en 5 min post-deploy
```

15.5. Feature Flags

Toda funcionalidad nueva potencialmente disruptiva se libera detrás de un feature flag. Esto desacopla deploy de release.

- **Activación gradual:** 1 % → 10 % → 50 % → 100 % de usuarios.
- **Por venue / por partner / por país:** permite testear con un Partner cooperador.
- **Kill switch:** apagar instantáneamente sin deploy.
- **Limpieza obligatoria:** todo flag tiene fecha de vencimiento y dueño. PRs de retiro obligatorios al expirar.

16. Reglas de Oro del Desarrollo

Estas son las directrices innegociables. Su violación, aún si pasa la revisión humana, debe ser detectada por linters automáticos. Lo que no se automatiza, se olvida.

Regla 1: El Meridiano Cero

Todo timestamp en backend y BD se almacena en UTC. Sin excepciones. El frontend es el único autorizado para formatear a hora local. Los operadores SQL como NOW() están prohibidos en migraciones; usar UTC_TIMESTAMP().

Regla 2: Claridad Multi-Cancha

Nunca mostrar 'Reserva 19:00' a secas. Siempre 'Cancha 2 - Sintético, 19:00'. La ambigüedad textual genera conflictos físicos en el local y reclamos al Staff.

Regla 3: UI de Cobranza Inequívoca

En la PWA del Staff, el saldo pendiente de cobro debe ocupar el área visual dominante: tipografía no menor a 36px, color rojo, sin distracciones cerca. Una cobranza fallida cuesta más que cualquier pixel.

Regla 4: Cero Cálculos Espaciales en Memoria

Prohibido SELECT * y filtrar por distancia en PHP/JS. Toda búsqueda por proximidad se filtra en SQL con ST_Distance_Sphere y un índice SPATIAL. Una violación de esta regla derribará el sistema con 50 venues activos.

Regla 5: Doble Validación de Pagos

Ningún booking pasa a reserved porque el frontend diga 'pago exitoso'. La única fuente de verdad es la confirmación asíncrona y firmada del webhook contra el backend, validada por HMAC. Confiar en el cliente es la causa número uno de fraude en marketplaces.

Regla 6: Idempotencia por Defecto

Toda operación que mute estado debe ser idempotente. Cliente reintenta mismo Idempotency-Key = mismo resultado, no duplicado. La red es poco confiable y el cliente puede no recibir tu 200; el reintento es legítimo.

Regla 7: Deny by Default

Toda autorización es denegada por defecto y debe ser explícitamente concedida por una Policy. Un endpoint sin Policy asociada falla la review automática. La autorización es un opt-in, no un opt-out.

Regla 8: Separación Estricta de Capas

Domain no importa Laravel. Application no toca Eloquent. Infrastructure no contiene reglas de negocio. La regla se cumple en CI con deptrac. Una violación bloquea el merge.

Regla 9: Construir para el Cambio

Toda dependencia externa (pasarela, mensajería, mapas) está detrás de una interfaz (Port). Cambiar de Mercado Pago a Stripe es escribir un nuevo adaptador, no reescribir el dominio. Esta es la diferencia entre un equipo que construye y un equipo que demuele y reconstruye.

Regla 10: Lo que no se mide no existe

Toda funcionalidad nueva en producción tiene logs, métricas y al menos una alerta accionable. Sin observabilidad no hay producción. Sin runbook no hay alerta.

17. Gobernanza del Documento y del Código

17.1. Cómo Cambia este Documento

Este documento es vinculante. Modificar una sección requiere:

8. Pull Request en el repositorio docs/ con la propuesta de cambio.
9. Aprobación de al menos 2 personas: el Tech Lead y, según la sección, Product Owner o Security Lead.
10. Actualización de la tabla 'Historial de Revisiones'.
11. Comunicación al equipo (canal #arquitectura) antes de fusionar.

17.2. Decisiones Arquitectónicas (ADRs)

Toda decisión arquitectónica relevante se documenta como ADR (Architecture Decision Record) en /docs/adr/NNNN-titulo.md con la siguiente estructura: Contexto, Decisión, Consecuencias, Alternativas Consideradas. Las ADRs son inmutables: una decisión nueva no edita la anterior, la supersede.

17.3. Definition of Ready (DoR) para una historia

- Tiene criterios de aceptación verificables.
- Identifica RFs y RNFs afectados.
- Identifica si afecta a una regla de negocio (RN).
- Identifica métricas de observabilidad nuevas o modificadas.
- Estimación realizada por al menos 2 personas.

17.4. Definition of Done (DoD) para una historia

- Código mergeado con todas las verificaciones de CI en verde.
- Tests escritos: unitarios, integración cuando corresponda.
- Documentación de API actualizada (OpenAPI).
- Métricas y alertas configuradas si aplica.
- Probado en staging con escenario realista.
- Feature flag definido si la funcionalidad es disruptiva.
- Runbook actualizado si introduce alerta nueva.

18. Anexos

18.1. Anexo A: Convenciones de Nombres

Elemento	Convención	Ejemplo
Tablas	plural, snake_case	shift_logs, booking_slots
Columnas	snake_case	starts_at, paid_advance
FKs	{tabla_singular}_id	venue_id, user_id
Índices	idx_{tabla}_{cols}	idx_slots_field_state_starts
Único	uk_{tabla}_{cols}	uk_slot_field_start
FKs (constraint)	fk_{tabla}_{ref}	fk_bookings_venue
Clases PHP	PascalCase	CreateBookingUseCase
Métodos PHP	camelCase	calculatePendingBalance()
Variables PHP	camelCase	\$pendingBalance
Endpoints REST	kebab-case en path, snake_case en body	/staff/walk-ins, { is_walk_in }
Eventos de dominio	PascalCase en pasado	BookingConfirmed, SlotReleased
Use Cases	VerboNombreUseCase	CheckInUseCase, MarkNoShowUseCase
Migraciones	yyyy_mm_dd_HHMMSS_descripcion.php	2026_05_01_120000_create_bookings_table.php

18.2. Anexo B: Lista de Verificación Pre-Producción

Antes de poner una feature en producción, recorrer esta lista:

- **Tests:** unit, integration, E2E pertinentes en verde.
- **Idempotencia:** verificada en endpoints de mutación.
- **Autorización:** Policy implementada y testada para cada actor.
- **Validación:** FormRequest con todas las reglas; invariantes en Domain.
- **Logs:** puntos clave logueados sin PII.

- **Métricas:** RED expuestas; métricas de negocio si aplica.
- **Alertas:** umbrales y runbook.
- **Documentación:** OpenAPI actualizado, ADR si decisión arquitectónica.
- **Feature flag:** creado si es disruptivo.
- **Migraciones:** reversibles, sin LOCK prolongado.
- **Backwards compatibility:** API no rompe contratos vigentes.
- **Seguridad:** OWASP checklist revisado.

18.3. Anexo C: Recursos y Lecturas Recomendadas

- Vaughn Vernon, Implementing Domain-Driven Design (referencia para hexagonal y DDD).
- Sam Newman, Building Microservices (criterios de cuándo extraer servicios).
- Google SRE Book (<https://sre.google/books/>) — observabilidad y gestión de incidentes.
- Microsoft Cloud Design Patterns — circuit breaker, retry, bulkhead.
- OWASP API Security Top 10 — referencia obligatoria de seguridad de APIs.
- Documentación de Laravel Sanctum, Horizon, Octane — fuente oficial.
- Documentación oficial de la pasarela de pagos elegida — webhooks, reconciliación.

18.4. Anexo D: Cierre

*Este documento existe por una razón: **que el equipo construya una sola vez**. Las decisiones aquí descritas son contratos. Los desarrolladores no son obreros que demuelen y reconstruyen muros: son ingenieros que ejecutan un plan diseñado para soportar adversidades, escalar con dignidad, y permitir cambios sin colapso.*

La mejor arquitectura es aquella que no requiere reinención cada vez que aparece un caso nuevo. Ese es el estándar. Todo lo demás es deuda técnica acumulándose en silencio.

— FIN DEL DOCUMENTO —